

Szegedi Tudományegyetem
Informatikai Intézet

SZAKDOLGOZAT

Szabó Patrik Péter

2026

Szegedi Tudományegyetem
Informatikai Intézet

PecaPont: Horgász hely foglaló és információ szerző
webalkalmazás

Szakdolgozat

Készítette:

Szabó Patrik Péter

programtervező
informatikus szakos
hallgató

Témavezető:

Dr. Bilicki Vilmos

egyetemi docens

Szeged

2026

Feladatkiírás

A horgászok számára jelenleg nehézkes egy helyen megbízható információt találni tavakról, versenyekről, hírekről és horgász helyekről. Az információk gyakran szétszórtnak, különböző weboldalakon vagy közösségi média felületeken érhetők el, ami időigényessé és kényelmetlenné teszi a tervezést.

A PecaPont célja, hogy egy központi, könnyen használható digitális platformot biztosítson, ahol a felhasználók gyorsan hozzáférhetnek minden releváns horgászati információhoz.

A feladat egy olyan webalkalmazás elkészítése Angular^[1] alapú keretrendszer használatával, ahol a felhasználók könnyedén elérnek minden releváns információt. Hozzáférnek vízterületek listájához, azoknak a részletes leírásával, tájékozódhatnak aktualitásokról, tudomást szerezhetnek versenyekről és megoszthatják sikereiket a közösséggel.

Tartalmi összefoglaló

Téma megnevezése:

PecaPont: Horgász hely foglaló és információ szerző webalkalmazás

A megadott feladat megfogalmazása:

Egy olyan webalkalmazás készítése, ahol a felhasználók könnyedén eligazodnak és megszerzik a kellő információt a horgászatuk megtervezéséhez.

Megoldási mód:

A feladat abszolválásához az Angular^[1] keretrendszert és Firebase^[2] felhő alapú szolgáltatásait használom.

Alkalmazott eszközök, módszerek:

A fejlesztés Visual Studio Code^[3]-al zajlik. A munka során Angular^[1] keretrendszert használok Typescript, HTML, SCSS nyelvekkel és Firebase^[2] felhő alapú szolgáltatásaival.

Elért eredmények:

A webalkalmazás Firebase^[2] hosting szolgáltatással elérhető az interneten, a forráskód és dokumentáció megtalálható a publikus GitHub^[4] repositoryban.

A webalkalmazás reszponzív megvalósításának köszönhetően nemcsak asztali számítógépen vagy laptopon, hanem mobil eszközökön is használható. Ennek köszönhetően széleskörűen elérhetőek a szükséges információk a horgászok számára.

Kulcsszavak:

Angular^[1], Firebase^[2], horgászat, foglalási rendszer, webalkalmazás

Tartalomjegyzék

Feladatkiírás	1
Tartalmi összefoglaló	2
Bevezetés	6
1. Piackutatás.....	8
1.1. Kérdőíves felmérés eredményei	9
1.2. Potenciális versenytársak	11
Horgaszhelyek.hu^[9].....	11
Vampire Fishing Trips^[10]	12
FishMap^[11].....	12
1.3. Összegzés.....	12
2. Funkcionális specifikáció.....	13
2.1. Funkcionális követelmények	13
2.2. Nem funkcionális követelmények	14
2.3. Használati eset diagram	14
2.4. Felhasználói szerepkörök	16
2.4.1. Vendég felhasználó	16
2.4.2. Bejelentkezett felhasználó	16
2.4.3. Újságíró	16
2.4.4. Szervező	16
2.4.5. Tókezelő	16
2.4.6. Adminisztrátor	17
3. Tervezett megjelenés	17
3.1. Főoldal.....	18
3.2. Horgászvizek oldal	18
3.3. Tó részletes oldal	19

3.4. Hírek oldal	19
3.5. Hír részletes oldal.....	19
3.6. Hírszerkesztő oldal.....	20
3.7. Versenyek oldal.....	20
3.8. Verseny részletes oldal	20
3.9. Galéria oldal	20
3.10. Profil oldal	20
3.11. Bejelentkezés és Regisztráció oldal.....	20
3.12. Admin felület	21
3.13. Tőkezelői felület.....	21
4. Felhasznált technológiák	22
4.1. Angular ^[1]	22
4.2. Firebase ^[2]	22
4.3. GitHub ^[4]	22
4.4. ChatGPT ^[5]	22
4.5. Gemini ^[6]	23
4.6. AngularFire ^[7]	23
4.7 Draw.io ^[8]	23
5. Architektúra	23
5.1. Frontend.....	24
5.2. Backend.....	24
6. Belső felépítés	24
6.1. Komponensek (Component)	25
6.2 Szolgáltatások (Service-ek).....	25
6.3. Interfészek (Interface)	25
6.4. Konfigurációs állományok (Config)	26

6.5. Routing.....	27
7. Biztonság.....	29
8. Adatmodell.....	32
8.1. Tavak (Lakes)	32
8.2. Galéria (Gallery)	32
8.3. Hírek (News).....	33
8.4. Versenyek (Events).....	33
8.5. Felhasználók (Users)	33
8.6. Foglalások (Bookings).....	33
8.7. Értékelések (Reviews)	34
9. A rendszer magasszintű folyamatai	34
10. Fontosabb kódrészletek	36
10.1. Firebase^[2] Authentication és felhasználókezelés	36
10.2. Jogosultságkezelés	37
10.3. Foglalási ütközések és elérhetőség ellenőrzése	38
10.4. Firebase^[2] Storage képfeltöltés	39
11. Mesterséges intelligencia használata a fejlesztés során	39
12. Tesztelés és validáció	42
13. Tapasztalatok, továbbfejlesztési lehetőségek	43
Irodalomjegyzék.....	46
Nyilatkozat.....	47
Elektronikus mellékletek.....	48
Mellékletek.....	49

Bevezetés

Az informatika és az internetes technológiák fejlődésének köszönhetően az információhoz való hozzáférés jelentősen átalakult az elmúlt évtizedekben. Ez a folyamat a horgászat világát sem kerülte el: egyre több horgászvíz üzemeltetője jelenik meg különböző online platformokon, elsősorban közösségi média felületeken. Ezek az oldalak azonban gyakran eltérő struktúrában, különböző részletességgel és nem egységes formában tartalmazzák az információkat, ami megnehezíti azok hatékony feldolgozását és összehasonlítását.

A horgászok jelentős része emiatt több forrásból próbál tájékozódni: közösségi média csoportokban, fórumokon, illetve más felhasználók beszámolóira támaszkodva gyűjtenek információt egy-egy horgászvízről. Ezek az információk azonban sok esetben szubjektívek, hiányosak vagy elavultak lehetnek, így nem minden esetben nyújtanak megbízható alapot a döntéshozatalhoz. A jelenlegi helyzet tehát egy olyan problémát vet fel, amelyben az információk szétszórtsága és változó minősége megnehezíti a horgászok számára a gyors és pontos tájékozódást.

A dolgozat célja egy olyan webalkalmazás, a PecaPont megtervezése és megvalósítása, amely egységes keretbe foglalja a horgászvizekkel kapcsolatos információkat, és egy központi platformot biztosít a felhasználók számára. Az alkalmazás célja, hogy lehetővé tegye a releváns adatok – például szabályok, módszerek, fogási tapasztalatok, értékelések és egyéb fontos információk – egyszerű és gyors elérését, ezáltal támogatva a horgászok döntéshozatalát.

Fontos szempont volt a webalkalmazás egyszerű és gyors használhatósága, hogy a felhasználók könnyedén eligazodhassanak az elérhető tartalmak között. Ennek érdekében a rendszer aloldalakkal épül fel, amelyek támogatják az információk kategorizálását, így a horgászok gyorsan megtalálják a számukra releváns adatokat.

A rendszer nem csupán statikus információk megjelenítésére szolgál, hanem közösségi funkciókat is integrál. A felhasználók lehetőséget kapnak arra, hogy megosszák saját élményeiket, beszámolóikat és fogásaikat, valamint visszajelzéseket adjanak mások bejegyzéseire. Ez hozzájárul egy aktív, egymást segítő közösség kialakulásához, ahol a tapasztalatcsere révén értékes, naprakész információk keletkeznek.

A rendszer további fontos funkciója a horgászhelyek online foglalásának támogatása. A felhasználók lehetőséget kapnak arra, hogy előre lefoglalják a kiválasztott horgászhelyeket, valamint nyomon követhessék foglalásaik állapotát. A foglalási

rendszer célja a szervezettebb és kényelmesebb horgászati tervezés támogatása mind a horgászok, mind a vízterületek kezelői számára.

A webalkalmazás tervezése során kiemelt szempont volt a felhasználóbarát működés és az átlátható felépítés. Ennek érdekében a rendszer több oldalból épül fel, kategóriára lebontva, így a felhasználók gyorsan megtalálhatják a számukra releváns tartalmakat. Emellett fontos célkitűzés volt a reszponzív kialakítás, amely biztosítja, hogy az alkalmazás különböző eszközökön – például mobiltelefonon, tableten vagy asztali számítógépen – egyaránt hatékonyan használható legyen.

A rendszer kialakítása során szerepkör alapú működés került alkalmazásra, amely lehetővé teszi az eltérő jogosultságok kezelését. A különböző szerepkörök – például adminisztrátor, újságíró, szervező és tókezelő – eltérő funkciókhoz férhetnek hozzá, ezáltal biztosítva a rendszer strukturált és biztonságos működését. Emellett a felhasználók értékeléseket és véleményeket is közzétehetnek az egyes horgászhelyekről, amelyek támogatják a közösségi alapú információmegosztást.

Összességében a PecaPont webalkalmazás egy olyan komplex megoldást kínál, amely egyesíti az információgyűjtést és a közösségi interakciókat, ezáltal hozzájárulva a horgászok számára elérhető információk minőségének javításához és könnyebb hozzáférhetőségéhez.

1. Piackutatás

A szakdolgozat készítésének kezdeti szakaszában piackutatást végeztem annak érdekében, hogy felmérjem a célcsoport igényeit, valamint a már létező megoldásokat és potenciális versenytársakat. A kutatás célja az volt, hogy meghatározható legyen, milyen funkciók és szolgáltatások szükségesek egy olyan webalkalmazás kialakításához, amely valódi értéket nyújt a horgászok számára.

A vizsgálat során több olyan weboldalt és webalkalmazást elemeztem, amelyek működési elve vagy célja részben vagy egészben hasonlít a PecaPont koncepciójához. Az elemzés során figyelembe vettem az egyes rendszerek funkcionalitását, felhasználói felületét, valamint az általuk kínált szolgáltatások körét. Külön figyelmet fordítottam arra, hogy ezek az alkalmazások milyen módon kezelik a horgászvizekkel kapcsolatos információkat, illetve milyen közösségi funkciókat biztosítanak.

A piackutatás másik fontos eleme egy anonim kérdőíves felmérés volt, amelynek célja a horgászok szokásainak, információszerzési módjainak és elvárásainak feltérképezése volt. A kérdőív segítségével adatokat gyűjtöttem többek között arról, hogy a felhasználók milyen platformokon keresnek információt, mennyire tartják megbízhatónak az ott talált tartalmakat, valamint milyen funkciókat tartanak hasznosnak egy ilyen jellegű alkalmazásban.

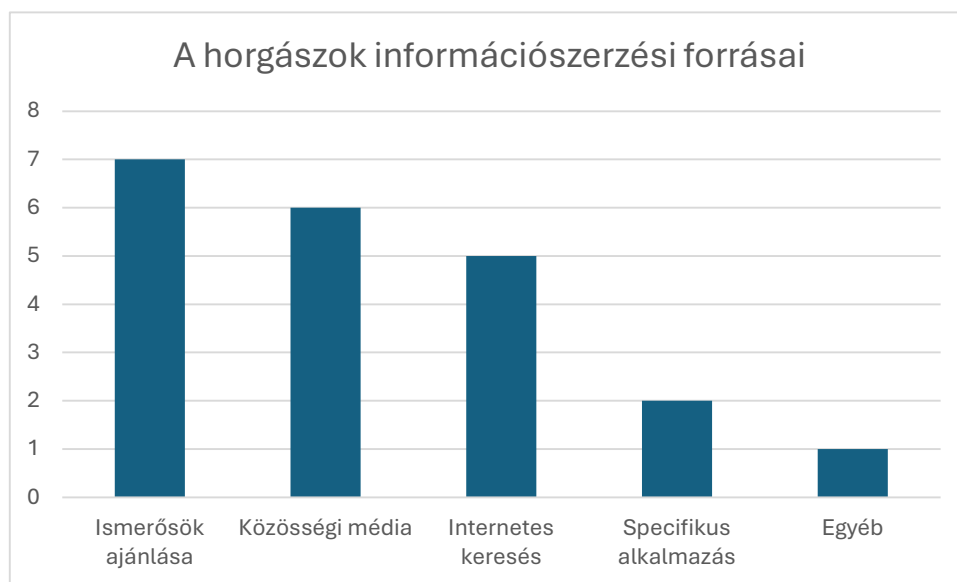
A kutatás eredményei hozzájárultak a PecaPont webalkalmazás követelményeinek meghatározásához, és alapul szolgáltak a rendszer tervezése során meghozott döntésekhez. Az így nyert információk segítségével lehetőség nyílt egy olyan alkalmazás kialakítására, amely jobban illeszkedik a felhasználói igényekhez, és hatékonyabban kezeli a horgászattal kapcsolatos információkat.

1.1. Kérdőíves felmérés eredményei

A piackutatás részeként egy anonim kérdőíves felmérést végeztem, amelynek célja a horgászok szokásainak, információszerzési módszereinek, valamint egy lehetséges webalkalmazással kapcsolatos elvárásainak feltérképezése volt. A kérdőívet különböző online felületeken tettem közzé, így a válaszadók eltérő korosztályból és tapasztalati háttérrel rendelkeztek.

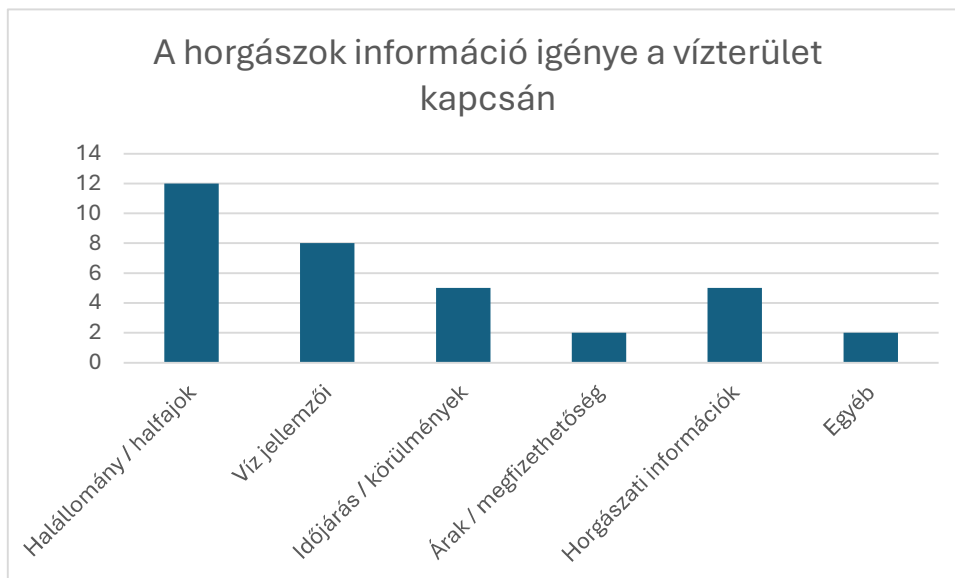
A kitöltők többsége férfi volt, és több korcsoport is képviseltette magát, ami lehetővé tette különböző felhasználói igények vizsgálatát. A válaszadók jelentős része több éves, esetenként évtizedes horgászati tapasztalattal rendelkezik, ami arra utal, hogy a célcsoport nem kizárólag kezdő horgászokból áll. Ennek megfelelően egy ilyen alkalmazásnak a tapasztaltabb felhasználók igényeit is ki kell szolgálnia.

Az információszerzési szokások vizsgálata során megállapítható volt, hogy a horgászok jelenleg több, egymástól független forrásból tájékozódnak. A leggyakoribb információforrások közé tartoznak az ismerősök ajánlásai, a közösségi média felületek, valamint az internetes keresések. Ez a gyakorlat megerősíti azt a problémát, hogy az információk szétszórtnak, nem egységes formában érhetők el.



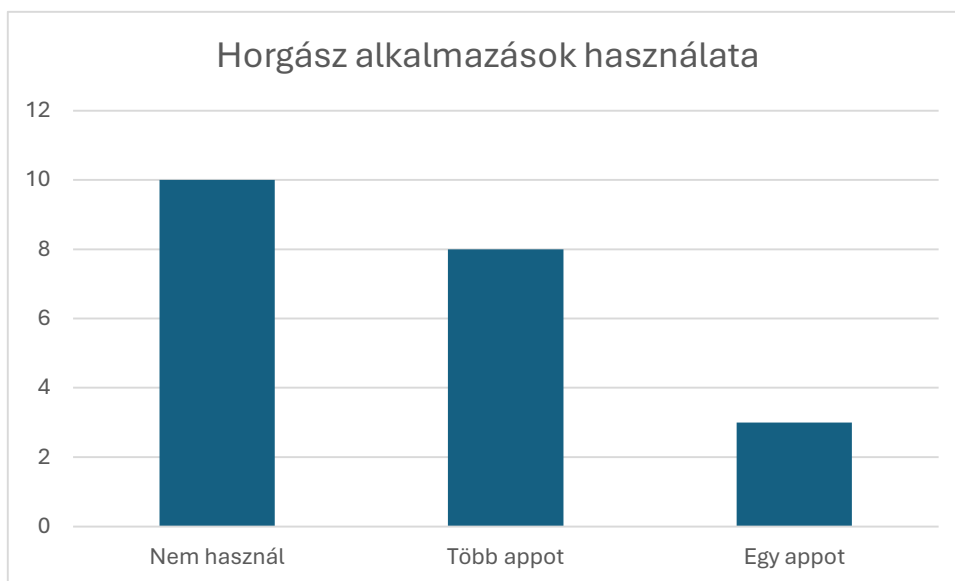
1. ábra a kérdőíves felmérés eredménye információszerzés kérdéskörben.

A válaszok alapján a horgászok számára kiemelten fontos információk közé tartozik a halállomány, a víz jellemzői, az árak, valamint az aktuális körülmények, például az időjárás vagy a víz állapota. Ezek az adatok alapvetően befolyásolják a horgászhely kiválasztását, így egy ilyen jellegű alkalmazásban központi szerepet kell, hogy kapjanak.



2. ábra a kérdőíves felmérés eredménye információigény témakörben

A jelenleg használt digitális megoldások vizsgálata során kiderült, hogy sok felhasználó nem használ kifejezetten horgászatra specializált alkalmazást, vagy több különálló platformot vesz igénybe (például időjárás-alkalmazásokat és közösségi oldalakat). Ez tovább erősíti egy egységes, több funkciót integráló rendszer iránti igényt.



3. ábra a kérdőíves felmérés eredménye alkalmazás használat témakörben

A válaszadók többsége pozitívan fogadta egy olyan webalkalmazás ötletét, amely egy helyen biztosítja a horgászvizekkel kapcsolatos információkat, valamint lehetőséget

ad a tapasztalatok és fogások megosztására. Többen kiemelték, hogy hasznosnak tartanák a részletes tóinformációkat, a közösségi visszajelzéseket, valamint az esetleges helyfoglalási lehetőségeket is.

A felmérés során az is megfigyelhető volt, hogy a felhasználók kerülnek a túlzott reklámokat, a félrevezető információkat és a nem átlátható, fizetős funkciókat. Ez fontos szempontként jelent meg a rendszer tervezése során.

Összességében a kérdőív eredményei alátámasztották a PecaPont webalkalmazás létjogosultságát. A válaszok alapján egyértelmű igény mutatkozik egy olyan egységes platform iránt, amely megbízható, strukturált és könnyen hozzáférhető információkat biztosít a horgászok számára, miközben közösségi funkciókat is kínál.

1.2. Potenciális versenytársak

A piackutatás során nemcsak a felhasználói igények feltérképezése történt meg, hanem a meglévő, hasonló célú rendszerek vizsgálata is. Bár kifejezetten a PecaPont koncepciójával teljes mértékben megegyező webalkalmazás nem található, több olyan platform létezik, amelyek részben hasonló funkciókat látnak el.

A horgászok által leggyakrabban használt felületek közé tartoznak a közösségi média platformok, például a Facebook különböző csoportjai. Ezekben a csoportokban a felhasználók megoszthatják tapasztalataikat, fogásaikat, valamint információt kérhetnek másoktól. Előnyük a gyors információcsere és a nagy felhasználói bázis, azonban az információk gyakran rendezetlenek, nehezen visszakereshetők, és megbízhatóságuk változó.

Léteznek továbbá kifejezetten horgászattal foglalkozó webalkalmazások, amelyek lehetőséget biztosítanak a fogások rögzítésére, statisztikák vezetésére, valamint közösségi funkciók használatára. Ezek az alkalmazások azonban sok esetben nem tartalmaznak részletes, helyspecifikus információkat a magyarországi horgászvizekről, illetve egyes funkcióik fizetősek.

Emellett különböző információs weboldalak is rendelkezésre állnak, amelyek egy-egy horgászvízről szolgáltatnak adatokat, például szabályzatokat vagy árakat. Ezek az oldalak azonban általában nem egységes rendszerben működnek, és ritkán kínálnak közösségi funkciókat vagy felhasználói visszajelzéseket.

Horgaszhelyek.hu^[9]

A [Horgaszhelyek.hu^{\[9\]}](http://Horgaszhelyek.hu) egy országos szintű adatbázis, amelynek célja, hogy a magyarországi horgászvizeket egy helyen tegye elérhetővé és összehasonlíthatóvá. Az

oldalon megtalálhatók a legfontosabb információk, például a halfajok, szabályok, szolgáltatások és árak, valamint felhasználói értékelések is.

Az oldal egyik legnagyobb előnye, hogy strukturált formában, jól áttekinthetően biztosít információkat a különböző vízterületekről, beleértve a halfajokat, szabályokat, szolgáltatásokat és árakat. Emellett keresési és szűrési lehetőségeket is kínál, ami megkönnyíti a felhasználók számára a megfelelő horgászhely kiválasztását. Ugyanakkor az oldal kevésbé fókuszál a közösségi interakciókra, és nem biztosít valós idejű adatokat, például aktuális fogási információkat, így inkább statikus információforrásként működik.

Vampire Fishing Trips^[10]

A Vampire Fishing Trips^[10] platform egy modernebb, technológiai megközelítést alkalmazó rendszer, amely elsősorban mobilalkalmazásként érhető el, és különböző digitális megoldásokkal támogatja a horgászokat.

Előnye, hogy közösségi és eseményalapú funkciókat is kínál, valamint a felhasználók aktivitására épít, ami növeli az interaktivitást.

Ellenben a rendszer kevésbé egységes felépítésű, és nem kifejezetten egy átfogó horgászhely-adatbázisra épül, így az információk nem minden esetben strukturáltak vagy könnyen összehasonlíthatók.

FishMap^[11]

A FishMap^[11] alkalmazás egy innovatív megoldás, amely térképes megjelenítéssel és valós idejű adatokkal segíti a horgászokat.

Az alkalmazás egyik legnagyobb erőssége, hogy GPS-alapú működésének köszönhetően lehetővé teszi a fogások rögzítését és azok vizualizálását, például hőterképek segítségével. Ez jelentősen megkönnyíti az aktív horgászhelyek azonosítását.

A rendszer főként a fogási adatokra koncentrál, és kevésbé nyújt részletes, strukturált információkat a horgászvizek egyéb jellemzőiről, például szabályokról vagy árakról, továbbá elsősorban mobilalkalmazásként érhető el.

1.3. Összegzés

Összességében megállapítható, hogy a vizsgált rendszerek különböző területeken erősek, azonban egyik sem kínál teljes körű, integrált megoldást. Egyes platformok strukturált adatokat biztosítanak, míg mások a közösségi vagy valós idejű információkra helyezik a hangsúlyt. A PecaPont célja ezen megközelítések egyesítése, egy olyan egységes rendszer

létrehozásával, amely egyszerre biztosít strukturált információkat, közösségi funkciókat és naprakész adatokat a felhasználók számára.

Funkció	Horgászhelyek.hu ^[9]	FishMap ^[11]	Vampire ^[10]	PecaPont
Helyadatok	igen	részleges	részleges	igen
Valós idejű fogások	nem	igen	igen	igen
Közösségi funkciók	részleges	igen	igen	igen
Egységes platform	részleges	nem	nem	igen

4. ábra a táblázat összefoglalja a vizsgált rendszerek főbb funkcióit, és összehasonlítja azokat a PecaPont webalkalmazással.

A táblázat alapján jól látható, hogy a jelenlegi megoldások különböző területeken nyújtanak erős funkcionalitást, azonban egyik sem kínál teljes körű megoldást. A Horgaszhelyek.hu^[9] elsősorban strukturált helyadatokat biztosít, de nem rendelkezik valós idejű információkkal. A FishMap^[11] és a Vampire^[10] platformok támogatják a valós idejű fogások megjelenítését és a közösségi funkciókat, ugyanakkor nem biztosítanak egységes, átfogó információs rendszert. Ezzel szemben a PecaPont célja, hogy egyetlen platformon egyesítse a hely adatokat, a közösségi funkciókat és a valós idejű információkat, így komplexebb megoldást nyújtva a felhasználók számára.

2. Funkcionális specifikáció

A piackutatás és a versenytársak elemzése alapján meghatározhatók azok a funkcionális és nem funkcionális követelmények, amelyeknek a PecaPont webalkalmazásnak meg kell felelnie. A kérdőíves felmérés eredményei rámutattak arra, hogy a horgászok számára kiemelten fontos a gyors, megbízható és strukturált információelérés, valamint a közösségi funkciók jelenléte. Emellett a versenytársak vizsgálata azt is megmutatta, hogy a jelenlegi megoldások nem kínálnak teljes körű, egységes platformot.

2.1. Funkcionális követelmények

A funkcionális követelmények a rendszer által biztosított konkrét szolgáltatásokat írják le.

A rendszernek biztosítania kell a horgászvizek adatainak megjelenítését, beleértve a halállományt, a víz jellemzőit, az árakat és a szabályokat. A felhasználóknak képesnek kell lenniük a horgászhelyek közötti keresésre és szűrésre különböző paraméterek alapján, például név, település vagy víztípus szerint.

A rendszernek támogatnia kell a felhasználói tartalmak kezelését. Ennek keretében a regisztrált felhasználók képesek fogásokat rögzíteni, bejegyzéseket létrehozni, valamint értékeléseket és hozzászólásokat írni a horgászhelyekhez.

A rendszernek biztosítania kell a közösségi interakciókat, beleértve a felhasználók közötti kommunikációt és a tartalmak megosztását. A felhasználóknak lehetőséget kell biztosítani más felhasználók tartalmainak megtekintésére.

A rendszernek képesnek kell lennie valós idejű vagy közel valós idejű információk megjelenítésére, például friss fogási adatok vagy aktuális környezeti információk esetében.

A rendszernek biztosítania kell a felhasználói fiókok kezelését, beleértve a regisztrációt, bejelentkezést, kijelentkezést, valamint a profiladatok módosítását.

A funkcionális követelmények közvetlenül a piackutatás eredményeiből és a versenytársak elemzéséből kerültek levezetésre.

2.2. Nem funkcionális követelmények

A nem funkcionális követelmények a rendszer működésének minőségi jellemzőit határozzák meg.

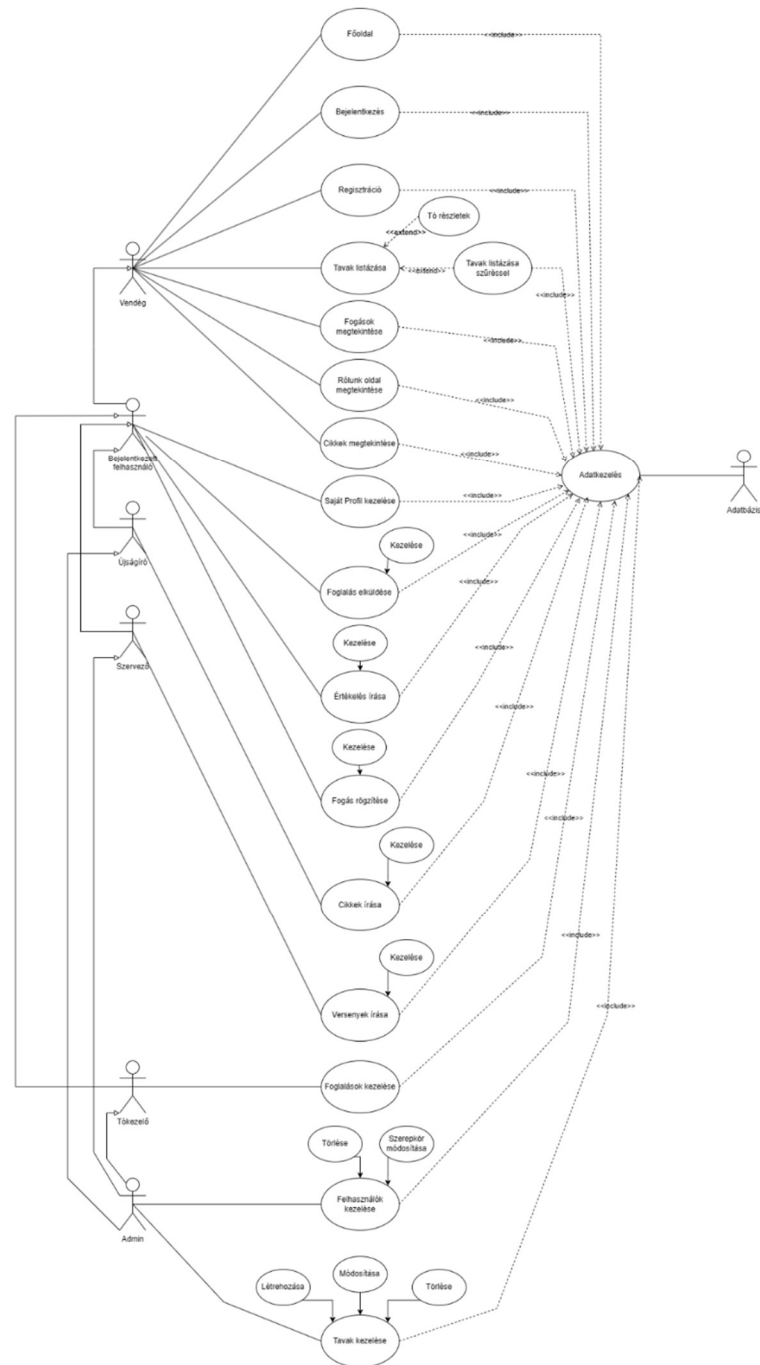
A webalkalmazásnak felhasználóbarát és könnyen kezelhető felülettel kell rendelkeznie, amely lehetővé teszi a felhasználók számára a gyors és hatékony navigációt. A rendszernek reszponzív kialakításúnak kell lennie, hogy különböző eszközökön, például mobiltelefonon, tableten és asztali számítógépen is megfelelően működjön.

A rendszernek biztosítania kell a gyors betöltési időt és a stabil működést, mivel a felhasználók elvárják a zökkenőmentes használatot. Emellett fontos a megbízhatóság és az adatok pontossága, különösen a felhasználók által generált tartalmak esetében.

Biztonsági szempontból a rendszernek védenie kell a felhasználói adatokat, és biztosítania kell a jogosultságkezelést, hogy a különböző funkciók csak megfelelő hozzáféréssel legyenek elérhetők.

2.3. Használati eset diagram

A rendszer működésének szemléltetésére használati eset diagram készült, amely bemutatja a rendszer és a felhasználók közötti kapcsolatokat, valamint a főbb funkcionálisokat.



5. ábra Az alkalmazás használati eset diagramja

A diagramon öt fő szereplő jelenik meg: a vendég felhasználó, a bejelentkezett felhasználó, az újságíró, a szervező és az adminisztrátor. A vendég felhasználók számára elsősorban a böngészési és keresési funkciók érhetők el, míg a bejelentkezett felhasználók további műveleteket végezhetnek, például fogásokat rögzíthetnek vagy hozzászólásokat írhatnak. Az újságíró cikket hozhat létre míg a szervező versenykiírásokat tehet közzé. Az adminisztrátor jogosult a rendszer adatainak kezelésére és a felhasználók adminisztrálására.

2.4. Felhasználói szerepkörök

2.4.1. Vendég felhasználó

A vendég felhasználók regisztráció és bejelentkezés nélkül is használhatják a webalkalmazást, azonban funkcionalitásuk korlátozott. Lehetőségük van a horgászhelyek böngészésére, keresésére és szűrésére, valamint a horgászhelyek adatlapjainak megtekintésére. Hozzáférnek a feltöltött fogásokhoz, valamint az alap oldalakhoz. Emellett lehetőségük van regisztrációra és bejelentkezésre.

2.4.2. Bejelentkezett felhasználó

A bejelentkezett felhasználók a rendszer teljes funkcionalitásához hozzáférnek. A horgászhelyek böngészése és megtekintése mellett lehetőségük van fogások rögzítésére és kezelésére, horgászhelyek foglalására és azok visszamondására, értékelések írására és kezelésére, valamint saját profiljuk kezelésére.

2.4.3. Újságíró

Az újságíró szerepkörrel rendelkező felhasználók speciális jogosultságokkal rendelkeznek a rendszerben. A bejelentkezett felhasználókhöz képest további lehetőségük van cikkek létrehozására és kezelésére. Ezek a cikkek információkat tartalmazhatnak például horgászati tippekről, beszámolókról vagy hírekről, amelyek a többi felhasználó számára is elérhetők.

2.4.4. Szervező

A szervező szerepkörrel rendelkező felhasználók jogosultak horgászversenyek létrehozására és kezelésére. A versenyek kiírása során megadhatják a szükséges adatokat, például a helyszínt, időpontot és a részvételi feltételeket. Ez a funkció lehetővé teszi a közösségi események szervezését és lebonyolítását a platformon belül.

2.4.5. Tókezelő

A tókezelő szerepkörrel rendelkező felhasználók speciális jogosultságokkal rendelkeznek a hozzájuk tartozó horgászhelyek kezelésére. Lehetőségük van a saját kezelésük alatt álló tavak adatainak módosítására, valamint az adott tavakhoz kapcsolódó foglalások megtekintésére és kezelésére. A tókezelők jóváhagyhatják vagy elutasíthatják a foglalási kérelmeket, ezáltal támogatva a foglalási rendszer működését és a horgászhelyek szervezett kezelését.

2.4.6. Adminisztrátor

Az adminisztrátor jogosult a rendszer adatainak kezelésére és statisztikák elérésére. Feladatai közé tartozik a felhasználók kezelése, a horgászhelyek adatainak karbantartása, a foglalások és értékelések moderálása, valamint a nem megfelelő tartalmak kezelése. Az adminisztrátor teljes hozzáféréssel rendelkezik a rendszer funkcióihoz és jogosultságaihoz.

3. Tervezett megjelenés

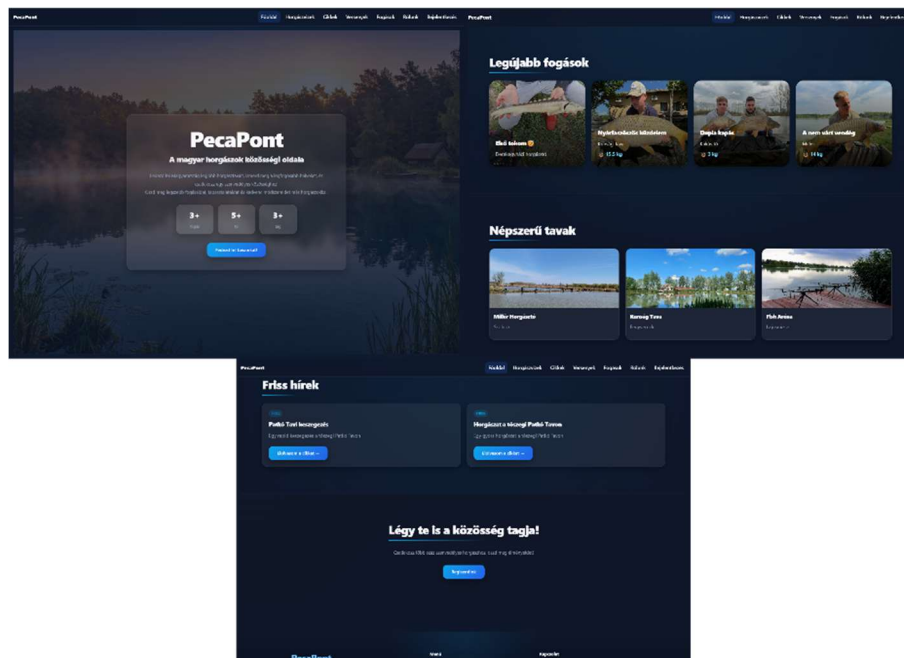
A webalkalmazás felhasználói felületének megtervezése során kiemelt szempont volt az átláthatóság, az egyszerű használhatóság, valamint a reszponzív megjelenés biztosítása. A képernyőtervek kialakítása során a modern webalkalmazások felépítését vettem alapul, különös figyelmet fordítva a felhasználói élményre.

Az alkalmazás egy egységes megjelenési móddal rendelkezik, amely a sötét témára épül, ugyanakkor különböző fényviszonyok mellett is jól használható. A felület kialakításánál a konzisztens színhasználat és az egységes komponensstruktúra volt meghatározó.

A reszponzív működés biztosítása érdekében a felület különböző képernyőméretekre lett optimalizálva, így az alkalmazás mobil eszközökön és tableteken is megfelelően használható. A fejlesztés során a különböző nézetek kialakítása során figyelembe vettem a kisebb kijelzők korlátait, például az elemek átrendeződését és a navigáció egyszerűsítését.

3.1. Főoldal

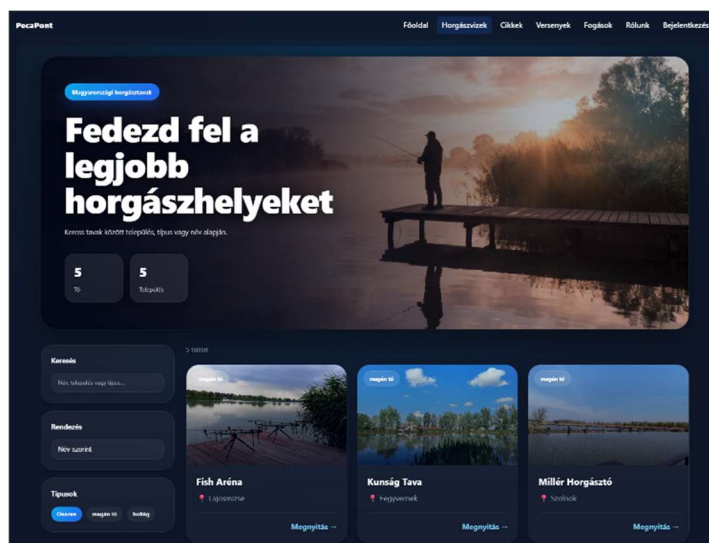
A főoldal kiemelt tartalmakat, gyors navigációs lehetőségeket és rövid információs blokkokat jelenít meg annak érdekében, hogy a felhasználók gyorsan elérhessék az alkalmazás legfontosabb funkcióit.



6. ábra A webalkalmazás főoldalának megjelenése

3.2. Horgászvizek oldal

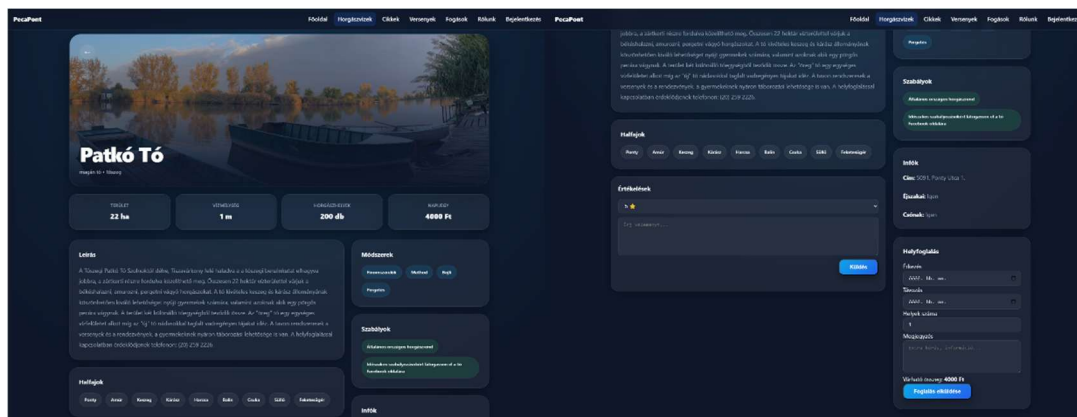
A horgászvizek oldalon a felhasználók a rendszerben szereplő horgászhelyek között böngészhetnek. Az elemek kártyás megjelenítésben jelennek meg, ahol az egyes tavak nevei és településük látható. A felhasználó egy adott elem kiválasztásával eljut a részletes nézetre.



7. ábra A horgászvizek oldal megjelenése

3.3. Tó részletes oldal

A tó részletes oldalán a kiválasztott vízterülethez tartozó részletes információk jelennek meg. A felhasználók megtekinthetik a tó jellemzőit, szabályait, halfajait, valamint értékeléseket és foglalási lehetőségeket is elérhetnek.



8. ábra Egy tavat bemutató oldal megjelenése

3.4. Hírek oldal

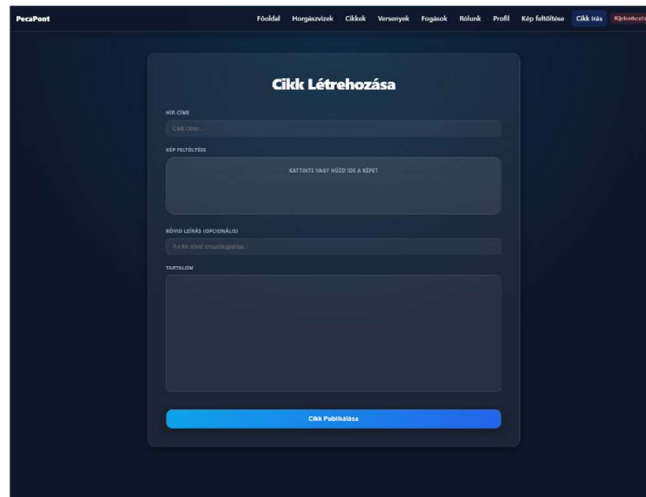
A hírek oldalon a horgászathoz kapcsolódó információk és bejegyzések jelennek meg. A hírek kártyás formában kerülnek megjelenítésre, és a felhasználó egy adott hír kiválasztásával részletesebb információkat tekinthet meg.

3.5. Hír részletes oldal

A hír részletes oldalán a felhasználók a kiválasztott cikk teljes tartalmát, képeit és a publikálás dátumát tekinthetik meg. Az oldal célja az információk könnyen olvasható és áttekinthető megjelenítése.

3.6. Hírszerkesztő oldal

A hírszerkesztő felület kizárólag jogosultsággal rendelkező felhasználók számára érhető el. Ezen az oldalon lehetőség van új hírek létrehozására, valamint meglévő tartalmak szerkesztésére.



9. ábra A cikkek létrehozására szolgáló felület megjelenése

3.7. Versenyek oldal

A versenyek oldalon a meghirdetett események kerülnek megjelenítésre. A felhasználók böngészhetnek a különböző versenyek között, és részletes információkat tekinthetnek meg az egyes eseményekről.

3.8. Verseny részletes oldal

A részletes verseny oldalon a felhasználók a kiválasztott esemény teljes leírásával, pontos részleteivel ismerkedhetnek meg.

3.9. Galéria oldal

A galéria oldalon a felhasználók által feltöltött fogások jelennek meg. A tartalmak minden felhasználó számára elérhetőek, míg új képet csak bejelentkezett felhasználók tölthetnek fel.

3.10. Profil oldal

A profil oldalon a felhasználók megtekinthetik saját adataikat, valamint módosíthatják azokat. Ez a felület kizárólag hitelesített felhasználók számára érhető el.

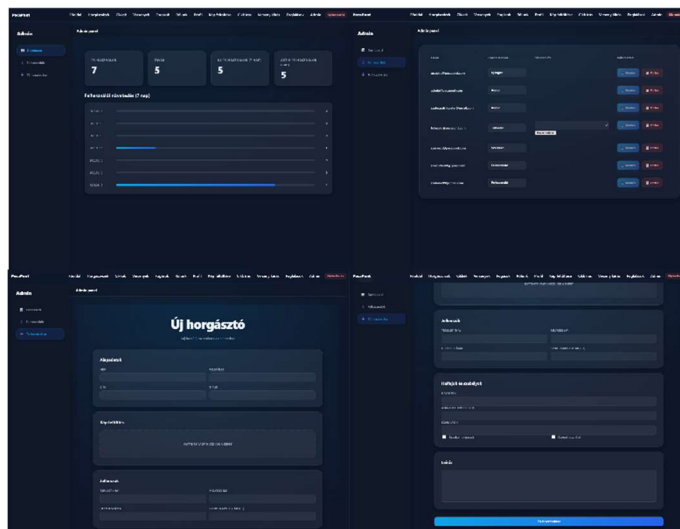
3.11. Bejelentkezés és Regisztráció oldal

A felhasználó számára lehetőség van regisztrációra és bejelentkezésre. A regisztráció során a felhasználók megadják a szükséges adatokat, amelyeket a rendszer a Firebase^[2]

Authentication segítségével kezel. A bejelentkezés sikeressége az adatok helyességétől függ.

3.12. Admin felület

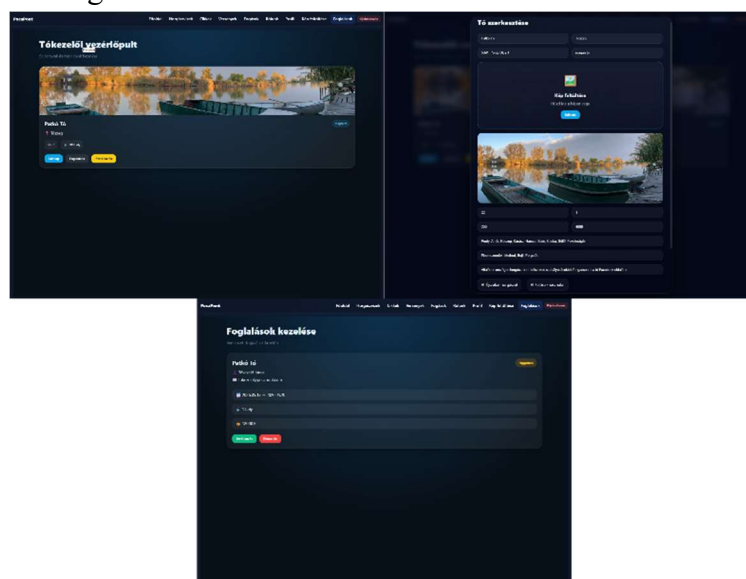
Az adminisztrációs felület kizárólag adminisztrátori jogosultsággal rendelkező felhasználók számára érhető el. Az admin felület lehetőséget biztosít a rendszer tartalmának kezelésére, például felhasználók vagy egyéb adatok módosítására, valamint rendelkezik egy statisztikai oldallal is.



10. ábra Az adminpanel 3 oldalának megjelenése

3.13. Tőkezelői felület

A tőkezelői felület kizárólag tőkezelő jogosultsággal rendelkező felhasználók számára érhető el. A felület lehetőséget biztosít a kezelésük alatt álló tavak áttekintésére, valamint az ezekhez tartozó foglalások kezelésére.



11. ábra A tőkezelő által elérhető felületek megjelenése

A tókezelők megtekinthetik a beérkező foglalási kérelmeket, valamint jóváhagyhatják vagy elutasíthatják azokat. A felület célja a foglalási folyamat egyszerűbb és átláthatóbb kezelése.

4. Felhasznált technológiák

4.1. Angular^[1]

Az Angular^[1] egy nyílt forráskódú, TypeScript alapú frontend keretrendszer, amelyet elsősorban komplex webalkalmazások fejlesztésére használnak. Az Angular^[1] komponens alapú felépítése lehetővé teszi az alkalmazás moduláris felépítését, ami megkönnyíti a karbantarthatóságot és az újrafelhasználhatóságot.

A PecaPont webalkalmazás fejlesztése során az Angular^[1] biztosította a kliensoldali logikát, a felhasználói felület kezelését, valamint az egyes funkciók strukturált megvalósítását.

4.2. Firebase^[2]

A Firebase^[2] egy felhőalapú platform, amely számos szolgáltatást kínál webes alkalmazások fejlesztéséhez. A projekt során több Firebase^[2] szolgáltatást is felhasználtam:

- Firestore (adatbázis): a horgászhelyek, felhasználók és fogások adatainak tárolására.
- Cloud Storage: képek és egyéb fájlok tárolására.
- Hosting: az alkalmazás publikálására.

A Firebase^[2] használatával lehetőség nyílt egy skálázható és valós idejű adatkezelést támogató rendszer kialakítására, amely gyors fejlesztést és egyszerű üzemeltetést biztosít.

4.3. GitHub^[4]

A GitHub^[4] egy verziókezelő rendszerre épülő platform, amely lehetővé teszi a forráskód nyomon követését és kezelését. A fejlesztés során a GitHub^[4] segítségével történt a projekt verziókezelése, amely biztosította a módosítások követhetőségét és a fejlesztési folyamat strukturálását.

4.4. ChatGPT^[5]

A fejlesztés során a ChatGPT^[5] mesterséges intelligencia alapú eszközt használtam különböző problémák megoldására, kódpéldák generálására, valamint dokumentációs

szövegek megfogalmazására. Az eszköz jelentősen hozzájárult a fejlesztési folyamat gyorsításához és a hatékonyabb munkavégzéshez.

4.5. Gemini^[6]

A Gemini^[6] AI egy mesterséges intelligencia alapú szolgáltatás, amely különböző szöveges műveletek elvégzésére alkalmas. A projekt során a Gemini^[6] AI használatával lehetőség nyílt szövegek generálására és feldolgozására, amely hozzájárulhat a felhasználói élmény javításához és az alkalmazás intelligens funkcióinak bővítéséhez.

4.6. AngularFire^[7]

Az AngularFire^[7] az Angular^[1] és a Firebase^[2] közötti integrációt biztosító hivatalos könyvtár. Segítségével egyszerűen megvalósítható a Firebase^[2] szolgáltatások, például az adatbázis-kezelés, hitelesítés és fájlkezelés használata Angular^[1] környezetben.

A projekt során az AngularFire^[7] könyvtárat használtam a Firestore adatbázis elérésére, az adatok lekérdezésére és módosítására, valamint a Firebase^[2] szolgáltatásokkal való kommunikáció megvalósítására.

4.7 Draw.io^[8]

A Draw.io^[8] (diagrams.net) egy online diagramkészítő eszköz, amely lehetővé teszi különböző típusú diagramok, például UML diagramok, folyamatábrák és rendszertervek elkészítését.

A szakdolgozat során a Draw.io^[8] segítségével készítettem el a használati eset diagramot, valamint a rendszer működését szemléltető további ábrákat.

5. Architektúra

A webalkalmazás architektúrája két fő részből áll: a frontend rétegből és a felhőalapú backend szolgáltatásokból. A rendszer nem hagyományos szerver–kliens architektúrát használ, hanem a Firebase^[2] platformra épülő Backend as a Service (BaaS) megoldást.

A frontend és a backend közötti kommunikáció a Firebase^[2] szolgáltatásain keresztül történik. A frontend közvetlenül éri el az adatbázist és egyéb backend funkciókat az AngularFire^[7] könyvtár segítségével, így nincs szükség külön backend szerver implementálására.

5.1. Frontend

A frontend résszel lép interakcióba a felhasználó a webalkalmazás használata során. Ez a réteg felelős a felhasználói felület megjelenítéséért és a felhasználói interakciók kezeléséért.

A frontend az Angular^[1] keretrendszer segítségével került megvalósításra, amely komponens alapú felépítést biztosít. Az alkalmazás különböző oldalakból és komponensekből épül fel, például navigációs menüből, listaoldalakból és adatlapokból.

A webalkalmazás struktúráját HTML sablonok határozzák meg, míg a megjelenésért és stílusért a SCSS felel. Az alkalmazás működéséhez szükséges logika TypeScript nyelven lett implementálva, amely kezeli a felhasználói eseményeket és az adatok lekérését.

5.2. Backend

A backend funkciókat a Firebase^[2] platform biztosítja, amely felhőalapú szolgáltatásokat kínál a webalkalmazás számára.

A Firestore adatbázis felelős az alkalmazás adatainak tárolásáért, beleértve a horgász helyeket, felhasználókat és fogásokat. A Cloud Storage szolgáltatás segítségével a felhasználók által feltöltött képek kerülnek tárolásra.

A Firebase^[2] Authentication biztosítja a felhasználók hitelesítését, lehetővé téve a regisztrációt és bejelentkezést. Az alkalmazás hosztolása szintén a Firebase^[2] Hosting szolgáltatáson keresztül történik.

A frontend közvetlenül kommunikál ezekkel a szolgáltatásokkal az AngularFire^[7] könyvtár segítségével, amely leegyszerűsíti az adatkezelést és a backend műveletek végrehajtását.

6. Belső felépítés

Az alkalmazás Angular^[1] keretrendszerrel készült, amely a fejlesztés során standalone komponens alapú felépítést használ. Az Angular^[1] újabb verzióiban (17+) a standalone komponensek kiemelt szerepet kapnak, így az alkalmazás modulok használata nélkül is felépíthető.

A standalone komponensek lehetővé teszik, hogy a szükséges komponenseket, direktívákat és pipe-okat közvetlenül importáljuk, ami egyszerűbbé és átláthatóbbá teszi az alkalmazás struktúráját.

6.1. Komponensek (Component)

A webalkalmazás több önálló komponensből épül fel, amelyek egymásba ágyazva alkotják a teljes felhasználói felületet. A komponensek tartalmazhatnak további komponenseket, direktívákat, pipe-okat és szolgáltatásokat (service-eket), ezáltal biztosítva az alkalmazás moduláris és újrafelhasználható felépítését.

A komponensek két fő csoportra oszthatók: Page komponensekre és újrafelhasználható UI komponensekre. A Page komponensek különálló oldalként jelennek meg az alkalmazásban, és URL-en keresztül érhetők el, míg az UI komponensek kisebb, több oldalon is felhasználható felületi elemeket valósítanak meg.

A PecaPont webalkalmazás számos Page és újrafelhasználható komponensből épül fel, amelyek együtt biztosítják az alkalmazás funkcionalitását és egységes megjelenését. A komponensek részletes listája és rövid leírása a 10. mellékletben található.

6.2 Szolgáltatások (Service-ek)

A szolgáltatások (service-ek) az alkalmazás üzleti logikáját, adatkezelését és jogosultságkezelését megvalósító osztályok. Feladatuk a komponensek és a Firebase^[2] alapú backend szolgáltatások közötti kommunikáció biztosítása. Az Angular^[1] dependency injection mechanizmusának köszönhetően a szolgáltatások több komponensben is újrafelhasználhatók.

A PecaPont webalkalmazásban külön szolgáltatások felelnek többek között a felhasználói hitelesítésért és jogosultságkezelésért, a tavak és foglalások kezeléséért, a hírek és versenyek kezeléséért, valamint a galéria és az értékelések működéséért. A szolgáltatások a Firebase^[2] Firestore, Authentication és Cloud Storage szolgáltatásaival kommunikálnak, és biztosítják az alkalmazás adatkezelési és üzleti logikai működését.

A létrehozott service-ek részletes listája és feladataik rövid leírása a 11. mellékletben található.

6.3. Interfészek (Interface)

Az interfészek segítségével meghatározható az alkalmazásban használt objektumok szerkezete, adattípusai és tulajdonságai. Az interfészek használata biztosítja az adatok egységes kezelését, valamint támogatja a TypeScript típusellenőrzési mechanizmusait, ezáltal növelve a kód átláthatóságát és megbízhatóságát.

A PecaPont webalkalmazásban létrehozott interfészek elsősorban a Firestore adatbázis kollekcioihoz és az alkalmazás üzleti logikájához kapcsolódnak. Az interfészek biztosítják, hogy a komponensek és szolgáltatások egységes adatstruktúrákkal dolgozzanak a tavak, foglalások, események, értékelések és galériaelemek kezelése során.

A projektben alkalmazott interfészek részletes listája és rövid leírása a 12. mellékletben található.

6.4. Konfigurációs állományok (Config)

Az alkalmazás működéséhez külön konfigurációs állományok kerültek kialakításra, amelyek célja az alkalmazásban használt állandó értékek, kategóriák és választható opciók központi kezelése. A konfigurációs állományok használata növeli a kód átláthatóságát és karbantarthatóságát, mivel az ismétlődő adatok egyetlen helyen módosíthatók.

A konfigurációs állományok segítségével az alkalmazás különböző komponensei és szolgáltatásai egységes adatforrást használnak, ezáltal biztosítva a konzisztens működést és megjelenítést.

- *catch-categories:*

A horgászati fogásokhoz kapcsolódó kategóriákat és halfajokat tartalmazó konfigurációs állomány. A rendszer külön kezeli például a békés halakat, ragadozó halakat, teríték fotókat és egyéb kategóriákat. Emellett tartalmazza az alkalmazásban választható horgásmódszereket, csalikat és napszakokat is. Ezek az adatok a fogások létrehozásánál és szűrésénél kerülnek felhasználásra.

- *event-categories:*

A horgászversenyekhez és eseményekhez kapcsolódó kategóriákat tartalmazó konfigurációs állomány. A rendszer előre definiált kategóriákat használ, például bojlis, feeder, method, úszós, gyerek és rendezvény típusú eseményeket. A konfiguráció biztosítja az események egységes kategorizálását és megjelenítését az alkalmazásban.

6.5. Routing



12. ábra Az alkalmazás routing struktúrája

A webalkalmazás felületei közötti navigáció megvalósítása az Angular^[1] keretrendszer Router moduljának segítségével történik. A Router feladata az URL-ek feldolgozása és azok megfelelő komponensekhez történő leképezése, amely lehetővé teszi a dinamikus, oldalújrátöltés nélküli navigációt. Ennek eredményeként az alkalmazás egyoldalas alkalmazásként (Single Page Application, SPA) működik.

A routing konfiguráció során az egyes funkcionális egységek külön útvonalakhoz kerültek hozzárendelésre. A kezdőoldal a gyökér útvonalon (/) érhető el, míg az egyéb tartalmak – például a tavak listája (/tavak) és az egyes tavak részletes nézete (/tavak/:id) – paraméterezett útvonalakon keresztül jelennek meg. A részletes tóoldalak támogatják a foglalási és értékelési funkciók megjelenítését is. Hasonló módon a hírek megjelenítése is támogatja az egyedi azonosítóval ellátott részletező nézeteket (/hirek/:id).

A felhasználói interakciókhoz kapcsolódó funkciók, mint például a regisztráció (/regisztracio), bejelentkezés (/bejelentkezes) és profilkezelés (/profil) szintén külön útvonalakon érhetők el. A galéria funkciók esetében elkülönül a tartalom megjelenítése (/galeria) és a képfeltöltés (/kepfeltoltes).

A foglalási rendszerhez kapcsolódó funkciók szintén külön útvonalakon érhetők el. A felhasználók számára biztosított foglalási oldalak lehetővé teszik a foglalások létrehozását és kezelését, míg a tókezelői felület külön útvonalakon keresztül érhető el. A manager-dashboard (/manager) oldal a tókezelő által kezelt tavak áttekintésére szolgál, míg a manager-bookings (/manager/:id) oldal egy adott tóhoz tartozó foglalások kezelését teszi lehetővé paraméterezett útvonalak használatával.

Az alkalmazás biztonságának biztosítása érdekében bizonyos útvonalak védelmét route guard-ok segítségével valósítottam meg. Az authGuard használatával kizárólag hitelesített felhasználók férhetnek hozzá olyan funkciókhoz, mint például a profil oldal, a képfeltöltés vagy a foglalási funkciók. A rendszer emellett több szerepkör alapú guardot is alkalmaz. Az adminGuard kizárólag adminisztrátori jogosultsággal rendelkező felhasználók számára biztosít hozzáférést az adminisztrációs felülethez. A newsGuard a hírszerkesztő funkciók elérését szabályozza, így csak megfelelő jogosultsággal rendelkező felhasználók hozhatnak létre vagy szerkeszthetnek híreket. Az eventsGuard hasonló módon a versenyekhez és eseményekhez kapcsolódó szerkesztői funkciókat védi. A managerGuard kizárólag a tókezelő szerepkörrel rendelkező felhasználók számára biztosít hozzáférést a tókezelői felülethez és a foglalások kezeléséhez. A guardok használata lehetővé teszi a szerepkör alapú hozzáférés-kezelés

megvalósítását, ezáltal növelve az alkalmazás biztonságát és biztosítva a különböző funkciók megfelelő elkülönítését.

Az adminisztrációs és kezelői modulok megvalósítása lazy loading technikával történt, amelynek célja az alkalmazás teljesítményének optimalizálása. Az admin felület (/admin) és annak aloldalai – például a felhasználókezelés (/admin/users) és az új tartalom hozzáadása (/admin/to-hozzaad) – csak akkor töltődnek be, amikor a felhasználó ténylegesen navigál ezekre az oldalakra. Hasonló módon a tókezelői felülethez tartozó oldalak is külön kerülnek betöltésre. Ez csökkenti a kezdeti betöltési időt és javítja a felhasználói élményt.

A routing konfiguráció végén egy ún. „wildcard” útvonal került definiálásra (**), amely minden nem létező útvonal esetén a kezdőoldalra irányítja vissza a felhasználót, ezáltal biztosítva az alkalmazás robusztus működését.

A routing rendszer kialakítása során fontos szempont volt az átlátható URL struktúra, a jogosultság alapú hozzáférés-kezelés és az alkalmazás teljesítményének optimalizálása.

7. Biztonság

A webalkalmazás biztonsági megoldásai több rétegben kerültek kialakításra, amelyek központi eleme a Firebase^[2] Cloud Firestore adatbázisra definiált biztonsági szabályrendszer. A Firestore lehetőséget biztosít finomhangolt hozzáférés-kezelésre, amely segítségével meghatározható, hogy az egyes kollekciókhoz milyen műveletek hajthatók végre különböző felhasználói szerepkörök esetén.

A rendszer megkülönbözteti a hitelesített (bejelentkezett) felhasználókat a vendégektől, ezáltal eltérő hozzáférési jogosultságokat biztosít számukra. A többféle szerepkör alkalmazásának köszönhetően a hitelesített felhasználók jogosultságai között is különbség tehető, amely lehetővé teszi a részletes jogosultságkezelés megvalósítását.

C = Létrehozás R = Olvasás U = Módosítás D = Törlés	Felhasználó	Fogások	Hírek	Versenyek	Értékelések	Foglalások
Vendég	CR	R	R	R	R	
Felhasználó	RUD	CRUD	R	R	CRUD	CRD
Újságíró	RUD	CRUD	CRUD	R	CRUD	CRD
Szervező	RUD	CRUD	R	CRUD	CRUD	CRD
Tőkezelő	RUD	CRUD	R	R	CRUD	CRUD
Admin	RUD	CRUD	CRUD	CRUD	CRUD	CRUD

13. ábra Kollektiókhoz és erőforrásokhoz való hozzáférés

A kollektiókhoz és erőforrásokhoz tartozó hozzáférési jogosultságokat a 13. ábra szemlélteti. A vendég felhasználók elsősorban olvasási jogosultsággal rendelkeznek, míg a hitelesített felhasználók a szerepkörüktől függően további műveletek végrehajtására is jogosultak, például tartalmak létrehozására és módosítására.

A Firestore biztonsági szabályok között kiemelt szerepet kap a felhasználók hitelesítettségének ellenőrzése, amely a következő feltétel segítségével történik: `request.auth != null`. A hitelesítés ellenőrzésén túl a rendszer további biztonsági mechanizmusokat is alkalmaz, amelyek a felhasználók szerepkörén és az adatok tulajdonjogán alapulnak. A Firestore szabályok között definiálásra kerültek olyan segédfüggvények, amelyek egyszerűsítik és egységesítik az ellenőrzések végrehajtását.

A Firestore biztonsági szabályrendszer teljes implementációja a projekt GitHub^[4] repositoryjában érhető el.

A szerepkör alapú jogosultságkezelés megvalósításához egy külön függvény került kialakításra, amely a felhasználó szerepkörét a users kollektióból olvassa ki, és összehasonlítja a szükséges jogosultsággal. Ennek segítségével meghatározható, hogy egy adott művelet végrehajtásához a felhasználó rendelkezik-e megfelelő jogosultsággal.

Az egyes kollektiókhoz tartozó szabályok eltérő működést valósítanak meg. A lates kollektió esetében az adatok olvasása minden felhasználó számára engedélyezett,

azonban módosítási műveleteket kizárólag adminisztrátori vagy a tóhoz tartozó tókezelői jogosultsággal rendelkező felhasználók végezhetnek.

A gallery kollekció esetében a rendszer biztosítja, hogy új tartalom létrehozása csak hitelesített felhasználók számára legyen elérhető, valamint ellenőrzi, hogy a létrehozott dokumentum a megfelelő felhasználóhoz tartozik. A módosítás és törlés kizárólag a tartalom tulajdonosa vagy adminisztrátor által hajtható végre, amely megakadályozza más felhasználók adatainak illetéktelen kezelését.

A bookings kollekció esetében a rendszer biztosítja, hogy foglalásokat kizárólag hitelesített felhasználók hozhassanak létre. A felhasználók kizárólag saját foglalásaikat érhetik el és kezelhetik, míg a tókezelők kizárólag a hozzájuk tartozó tavak foglalásainak kezelésére jogosultak. Az adminisztrátorok teljes hozzáféréssel rendelkeznek a foglalási adatokhoz.

A reviews kollekció a felhasználói értékelések és vélemények tárolására szolgál. A rendszer biztosítja, hogy új értékelést kizárólag hitelesített felhasználók hozhassanak létre. A módosítás és törlés kizárólag az értékelés tulajdonosa vagy adminisztrátor által hajtható végre.

A news és events kollekciók esetében a hozzáférés szerepkör-alapú módon kerül szabályozásra. Míg az adatok olvasása minden felhasználó számára engedélyezett, addig a létrehozás, módosítás és törlés csak meghatározott szerepkörökhöz kötött. A news kollekció kezeléséhez újságírói vagy adminisztrátori jogosultság szükséges, míg az events kollekció kezelése szervezői vagy adminisztrátori jogosultsághoz kötött.

A users kollekció kezelése kiemelt biztonsági szempontokat követ. A felhasználók kizárólag a saját adataikhoz férhetnek hozzá, és azok módosítása során nem változtathatják meg saját szerepkörüket. Ez a megkötés megakadályozza a jogosultsági szintek jogosulatlan módosítását.

A backend oldali biztonsági megoldásokat frontend oldali védelmi mechanizmusok is kiegészítik. Az Angular^[1] alkalmazás route guard-okat használ annak érdekében, hogy bizonyos oldalak kizárólag megfelelő jogosultsággal rendelkező felhasználók számára legyenek elérhetők. Az authGuard a hitelesített felhasználók hozzáférését ellenőrzi, míg az adminGuard az adminisztrátori jogosultság meglétét

vizsgálja. A newsGuard a hírszerkesztő felületek elérését szabályozza, az eventsGuard pedig a versenyekhez és eseményekhez kapcsolódó szerkesztői funkciókat védi. A managerGuard kizárólag a tőkezelő szerepkörrel rendelkező felhasználók számára biztosít hozzáférést a tőkezelői felülethez és a foglalások kezeléséhez.

Emellett az alkalmazás űrlapvalidációkat is alkalmaz, amelyek biztosítják a felhasználói adatok helyességét és teljességét.

8. Adatmodell

Adatok tárolására Google Firebase^[2] Cloud Firestore szolgáltatást alkalmaztam, míg a fájlok kezelésére a Firebase^[2] Storage szolgáltatás került felhasználásra.

A Cloud Firestore NoSQL típusú adatbázis, amelyben az adatok dokumentumok formájában kerülnek eltárolásra, és ezek kollekciókba szerveződnek. A relációs adatbázisokkal ellentétben a NoSQL nem használ hagyományos táblák közötti kapcsolatokat, hanem az adatok közötti kapcsolatok logikai szinten kerülnek megvalósításra. Az alkalmazás adatmodelljét a 13. melléklet mutatja be.

8.1. Tavak (Lakes)

A tavak kollekcióban a rendszerben megjelenített horgászhelyek adatai kerülnek eltárolásra. A tavakat minden felhasználó megtekintheti, azonban módosításuk adminisztrátori vagy a tóhoz tartozó tőkezelői jogosultsággal lehetséges. A dokumentumok tartalmazzák a tőkezelőhöz kapcsolódó adatokat is, például a kezelő azonosítóját és nevét.

Egy vízterülethez eltárolásra kerül annak neve, leírása, elhelyezkedése, valamint egyéb releváns adatok. A dokumentumok egyedi azonosítóval rendelkeznek, amely lehetővé teszi azok egyértelmű azonosítását.

8.2. Galéria (Gallery)

A galéria kollekcióban a felhasználók által feltöltött képek és azok leírásai kerülnek tárolásra. A tartalmak minden felhasználó számára elérhetőek, azonban feltöltés kizárólag bejelentkezett felhasználók számára engedélyezett.

Egy galéria elem tartalmazza a feltöltő felhasználó azonosítóját, a kép elérési útvonalát, valamint a képen szereplő fogás adatait. A képek fizikai tárolása a Firebase^[2] Storage szolgáltatásban történik. A galéria dokumentumok további adatokat is

tartalmazhatnak, például a halfaj típusát, a fogás súlyát, hosszát, az alkalmazott csalit és módszert.

8.3. Hírek (News)

A hírek kollekcióban az alkalmazásban megjelenő információs tartalmak találhatóak. A hírek minden felhasználó számára elérhetőek, azonban létrehozásuk és módosításuk szerepkörhöz kötött.

Egy hírhez eltárolásra kerül a cím, a tartalom, a létrehozás dátuma, valamint a szerző azonosítója és a létrehozás időpontja.

8.4. Versenyek (Events)

A versenyek kollekció tartalmazza a meghirdetett eseményeket. Ezek az adatok minden felhasználó számára elérhetőek, azonban létrehozásuk és módosításuk kizárólag megfelelő jogosultsággal rendelkező felhasználók számára engedélyezett.

Egy versenyhez eltárolásra kerül annak neve, időpontja, helyszíne, leírása, valamint a szervező azonosítója. Valamint verseny dokumentumok tartalmazza az esemény kategóriáját és a hozzá tartozó kép elérési útját is.

8.5. Felhasználók (Users)

A felhasználók kollekcióban a regisztrált felhasználók adatai kerülnek tárolásra. Az autentikációs adatokat a Firebase^[2] Authentication kezeli, míg a további felhasználói adatok ebben a kollekcióban kerülnek eltárolásra.

Egy felhasználóról eltárolásra kerül az egyedi azonosító (uid), név, e-mail cím, felhasználónév, szerepkör, valamint a létrehozás és az utolsó bejelentkezés időpontja.

8.6. Foglalások (Bookings)

A bookings kollekció a felhasználók által létrehozott foglalások adatait tárolja. A foglalások minden esetben kapcsolódnak egy adott horgász helyhez és felhasználóhoz.

Egy foglalás dokumentum tartalmazza a foglaló felhasználó azonosítóját és alapadatait, a kiválasztott tó azonosítóját és nevét, a foglalás kezdő és záró időpontját, a lefoglalt helyek számát, valamint a foglalás teljes árát.

A dokumentumok tárolják továbbá a foglalás aktuális állapotát is, például jóváhagyás alatt, jóváhagyva, elutasítva vagy törölve állapotot. A foglalások tartalmazhatják a tókezelő azonosítóját és a létrehozás időpontját is.

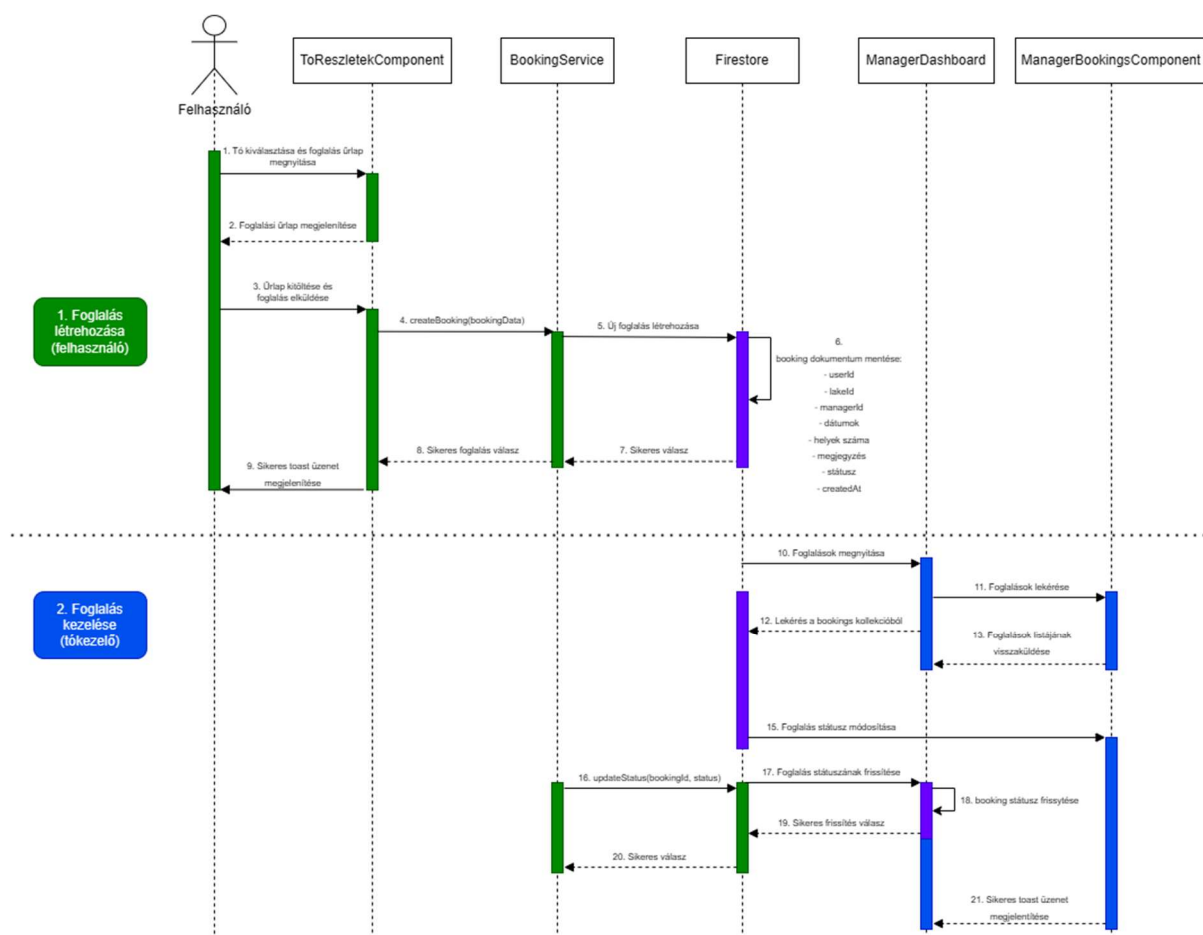
8.7. Értékelések (Reviews)

A reviews kollekció a felhasználók által létrehozott értékeléseket és véleményeket tárolja. Az értékelések kapcsolódhatnak egy adott horgászhelyhez, és lehetővé teszik a közösségi alapú visszajelzések megjelenítését.

Egy értékelés dokumentum tartalmazza a felhasználó azonosítóját, nevét, az adott pontszámot, a szöveges véleményt, valamint a létrehozás időpontját.

9. A rendszer magasszintű folyamatai

A webalkalmazás magasszintű folyamatai közé tartozik a foglalási rendszer működése, a hírek és versenyek kezelése, valamint a felhasználói tartalmak feltöltése és kezelése. A rendszer egyik legfontosabb folyamata a foglalások létrehozása és kezelése, amely több komponens és szolgáltatás együttműködésével valósul meg.



14. ábra Foglalási folyamat szekvencia diagramja

A 14. ábra a foglalási folyamat működését szemlélteti. A diagram bemutatja, hogyan történik egy új foglalás létrehozása a felhasználói felületen keresztül, valamint

annak eltárolása a Firestore adatbázisban a BookingService segítségével. A folyamat második része a tókezelői oldalon történő foglaláskezelést mutatja be. A tókezelő lekérheti a hozzá tartozó tavak foglalásait, majd módosíthatja azok állapotát, például jóváhagyhatja vagy elutasíthatja azokat. A státuszváltozások a Firestore adatbázisban kerülnek eltárolásra.

A foglalási folyamat során a felhasználó a kiválasztott tó részletes oldaláról kezdeményezheti a foglalást. A foglalási űrlap kitöltése után az alkalmazás ellenőrzi a megadott adatok helyességét és teljességét, majd a foglalási adatokat átadja a BookingService szolgáltatás megfelelő metódusának.

A szolgáltatás létrehozza az új foglalási dokumentumot a Firestore adatbázis bookings kollekciójában, amely tartalmazza a felhasználó, a kiválasztott tó és a foglalási időszak adatait. A rendszer a foglalást alapértelmezetten „jóváhagyás alatt” állapotban hozza létre.

A tókezelő a manager-dashboard felületen keresztül megtekintheti a hozzá tartozó tavakat, majd a manager-bookings oldalon kezelheti az adott tó foglalásait. A foglalások jóváhagyása vagy elutasítása során a BookingService módosítja a foglalás állapotát az adatbázisban.

A hírek és versenyek létrehozása hasonló működési elvet követ. A hírszerkesztő és versenyszerkesztő oldalak űrlapjai a megadott adatokat a megfelelő service-eknek adják át, amelyek létrehozzák vagy módosítják a dokumentumokat a Firestore adatbázisban. A képfeltöltések esetében a rendszer először a [Firebase^{\[2\]}](#) Storage szolgáltatásba tölti fel a fájlt, majd az elérési út bekerül az adatbázis dokumentumaiba.

A galéria feltöltési folyamat során a felhasználók képeket és fogási adatokat tölthetnek fel. A GalleryService kezeli a képfeltöltést és a kapcsolódó metaadatok eltárolását. A rendszer a feltöltött tartalmakat valós időben jeleníti meg a felhasználói felületen.

A műveletek végrehajtása után a webalkalmazás visszajelzést jelenít meg a felhasználó számára, például sikeres mentést vagy hibaüzenetet, majd szükség esetén automatikusan visszavigál a megfelelő oldalra.

10. Fontosabb kódrészletek

10.1. Firebase^[2] Authentication és felhasználókezelés

```

async register() {
  // 1. Üres mezők ellenőrzése
  if (!this.email || !this.password || !this.displayName || !this.username) {
    this.errorMessage = 'Kérjük, tölts ki minden mezőt!';
    return;
  }

  // 2. Jelszó komplexitás ellenőrzése (Regex)
  // Elvárás: min 8 karakter, legalább egy nagybetű, egy kisbetű és egy szám
  const passwordRegex = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d){8}$/;

  if (!passwordRegex.test(this.password)) {
    this.errorMessage = 'A jelszónak legalább 8 karakternek kell lennie, tartalmaznia kell kisbetűt, nagybetűt és számot!';
    return;
  }

  this.isLoading = true;
  this.errorMessage = '';
  this.cdr.detectChanges();

  try {
    // Átadjuk az adatokat a service-nek (Auth + Firestore mentés)
    await this.auth.register(this.email, this.password, this.displayName, this.username);
    console.log('Sikeres regisztráció!');
    this.router.navigate(['/']);
  } catch (error: any) {
    console.error('Regisztrációs hiba:', error.code);

    switch (error.code) {
      case 'auth/email-already-in-use':
        this.errorMessage = 'Ez az email cím már használatban van!';
        break;
      case 'auth/invalid-email':
        this.errorMessage = 'Érvénytelen email cím formátum!';
        break;
      case 'auth/weak-password':
        this.errorMessage = 'A megadott jelszó túl gyenge!';
        break;
      case 'auth/network-request-failed':
        this.errorMessage = 'Nincs internetkapcsolat!';
        break;
      default:
        this.errorMessage = 'Váratlan hiba történt a regisztráció során!';
    }
  } finally {
    this.isLoading = false;
    this.cdr.detectChanges();
  }
}

```

15. ábra A regisztrációs folyamatot kezelő kód

A regisztrációs folyamat megvalósítása során fontos szempont volt a felhasználói adatok ellenőrzése és a hibakezelés biztosítása. A regisztrációs komponens frontend oldali validációt alkalmaz annak érdekében, hogy a felhasználók csak megfelelő formátumú adatokat adhassanak meg.

A rendszer ellenőrzi, hogy minden szükséges mező kitöltésre került-e, valamint reguláris kifejezés segítségével validálja a jelszó komplexitását. A jelszónak legalább nyolc karakter hosszúságúnak kell lennie, továbbá tartalmaznia kell kisbetűt, nagybetűt és számot is.

Sikeres validáció után a komponens meghívja az AuthService regisztrációs metódusát, amely létrehozza a felhasználót a Firebase^[2] Authentication rendszerben, majd eltárolja a kapcsolódó adatokat a Firestore adatbázisban.

A rendszer kezeli a Firebase^[2] által visszaadott hibakódokat is, amelyek alapján felhasználóbarát hibaüzenetek jelennek meg például foglalt e-mail cím, hibás e-mail formátum vagy hálózati hiba esetén.

10.2. Jogosultságkezelés

```
export const adminGuard: CanActivateFn = async () => {
  const auth = inject(Auth);
  const firestore = inject(Firestore);
  const router = inject(Router);

  const user = auth.currentUser;

  if (!user) {
    return router.createUrlTree(['/bejelentkezés']);
  }

  const token = await user.getIdTokenResult(true);

  if (token.claims['admin']) {
    return true;
  }

  const userRef = doc(firestore, 'users', user.uid);
  const snap = await getDoc(userRef);

  if (snap.exists() && snap.data()['role'] === 'admin') {
    return true;
  }

  return router.createUrlTree(['/']);
};
```

16. ábra Az admin jogosultság meglétét ellenőrző kód

Az alkalmazás biztonságának növelése érdekében a védett oldalak elérését Angular^[1] route guard-ok segítségével valósítottam meg. Az adminGuard feladata annak ellenőrzése, hogy a felhasználó rendelkezik-e adminisztrátori jogosultsággal az adminisztrációs felület eléréséhez.

A guard működése során a rendszer először ellenőrzi, hogy létezik-e hitelesített felhasználó. Amennyiben nincs bejelentkezett felhasználó, a rendszer automatikusan a bejelentkezési oldalra irányítja vissza a felhasználót.

A jogosultság ellenőrzése két lépésben történik. Elsőként a Firebase^[2] Authentication tokenben tárolt custom claim értékek kerülnek ellenőrzésre, amely gyors jogosultságvizsgálatot tesz lehetővé. Amennyiben ez nem biztosít megfelelő eredményt, a rendszer a Firestore adatbázisból lekéri a felhasználó szerepkörét, és annak alapján dönt a hozzáférés engedélyezéséről.

A route guard használata biztosítja, hogy adminisztrációs funkciók kizárólag megfelelő jogosultsággal rendelkező felhasználók számára legyenek elérhetőek.

10.3. Foglalási ütközések és elérhetőség ellenőrzése

```
private isOverlapping(  
  from: Date,  
  to: Date,  
  bookingFrom: Date,  
  bookingTo: Date  
): boolean {  
  
  return (  
    from < bookingTo  
    &&  
    to > bookingFrom  
  );  
}  
  
async checkAvailability(  
  lakeId: string,  
  from: Date,  
  to: Date,  
  requestedPlaces: number,  
  maxPlaces: number  
): Promise<boolean> {  
  
  const occupied =  
    await this.getOccupiedPlaces(  
      lakeId,  
      from,  
      to  
    );  
  
  return (  
    occupied  
    + requestedPlaces  
  ) <= maxPlaces;  
}
```

17. ábra A foglaltságot és az ütközéseket ellenőrző kód

A foglalási rendszer egyik legfontosabb feladata annak ellenőrzése, hogy egy adott időszakban rendelkezésre állnak-e még szabad horgászhelyek. Ennek megvalósítására külön logika került kialakításra a BookingService szolgáltatásban.

Az isOverlapping() függvény feladata annak meghatározása, hogy két foglalási időintervallum átfedi-e egymást. Az ellenőrzés során a rendszer összehasonlítja az új foglalás kezdő és záró időpontját a már meglévő foglalások időintervallumaival.

A checkAvailability() metódus a Firestore adatbázisból lekéri az adott tóhoz tartozó aktív foglalásokat, majd kiszámítja az adott időszakban már lefoglalt helyek számát. A rendszer ezt követően ellenőrzi, hogy az új foglalás létrehozása nem lépi-e túl a tó maximális férőhelyszámát.

Ez a megoldás biztosítja, hogy a felhasználók ne tudjanak több helyet lefoglalni, mint amennyi ténylegesen rendelkezésre áll.

10.4. Firebase^[2] Storage képfeltöltés

```
const imageUrl =
  await this.galleryService
    .uploadImage(
      this.selectedFile,
      user.uid
    );

const postData = {
  ...this.form.value,
  water,
  weight:
    this.form.value.weight
      ? Number(
          this.form.value.weight
        )
      : null,
  length:
    this.form.value.length
      ? Number(
          this.form.value.length
        )
      : null,
  imageUrl,
  uid:
    user.uid,
  username,
  createdAt:
    Timestamp.now()
};

await this.galleryService
  .createPost(postData);
```

18. ábra A képfeltöltést kezelő kód

A galéria feltöltési folyamat során a felhasználók képeket és a hozzájuk tartozó fogási adatokat tölthetnek fel. A rendszer a képfájlokat a Firebase^[2] Cloud Storage szolgáltatásban tárolja, majd a feltöltött kép elérési útját eltárolja a Firestore adatbázisban létrehozott dokumentumban.

A feltöltési folyamat során a rendszer először feltölti a kiválasztott képet, majd lekéri annak URL címét. Ezt követően a felhasználó által megadott adatokból létrehoz egy összetett objektumot, amely tartalmazza a fogás adatait, a feltöltött kép elérési útját, valamint a feltöltő felhasználó azonosítóját és nevét.

A rendszer a numerikus mezők típuskonverzióját is elvégzi, valamint automatikusan rögzíti a feltöltés időpontját Timestamp segítségével.

11. Mesterséges intelligencia használata a fejlesztés során

A szakdolgozat készítése és a PecaPont webalkalmazás fejlesztése során mesterséges intelligencia alapú eszközöket is alkalmaztam a fejlesztési folyamat támogatására. A projekt során elsősorban a ChatGPT^[5] és a Gemini^[6] AI szolgáltatásokat használtam különböző technikai, dokumentációs és hibakeresési feladatokhoz. Az AI eszközök használata nem helyettesítette a saját fejlesztői munkát, hanem a fejlesztési folyamat támogatását és gyorsítását szolgálta.

A fejlesztés különböző szakaszaiban eltérő módon használtam az AI eszközöket. A projekt kezdeti tervezési szakaszában elsősorban ötletelésre és technológiai döntések támogatására alkalmaztam őket. Segítséget nyújtottak Angular^[1] és Firebase^[2] alapú architektúrák összehasonlításában, valamint bizonyos megvalósítási lehetőségek áttekintésében. A frontend fejlesztés során főként Angular^[1] standalone komponensekkel, route guard megoldásokkal, Firebase^[2] Authentication integrációval és Firestore adatkezeléssel kapcsolatos kérdésekben használtam őket.

A ChatGPT^[5] használata elsősorban programozási támogatásra, hibakeresésre és dokumentációs feladatokra irányult. Segítségével gyorsabban tudtam megoldást találni Angular^[1] és TypeScript hibákra, Firestore lekérdezések kialakítására, valamint Firebase^[2] Security Rules szabályok megírására. Az AI több esetben generált mintakódokat is, amelyeket a projekt saját igényeihez igazítottam és módosítottam. Különösen hasznosnak bizonyult a route guard-ok kialakításánál, az Observable alapú működés megértésénél, valamint a Firebase^[2] jogosultságkezelés implementálásánál.

A Gemini^[6] AI használata elsősorban szöveggenerálási és ötletelési feladatokhoz kapcsolódott. Segítséget nyújtott rövidebb leírások, dokumentációs szövegek és szakdolgozati fejezetek megfogalmazásában, valamint bizonyos funkciók továbbfejlesztési lehetőségeinek áttekintésében. Emellett a különböző felhasználói felületek és funkciók megnevezésének egységesítésében is támogatást nyújtott.

A mesterséges intelligencia eszközöket több konkrét fejlesztési feladathoz is felhasználtam. Ezek közé tartozott:

- Angular^[1] komponensek és service-ek mintakódjainak generálása,
- Firestore lekérdezések és security rules kialakítása,
- hibakeresés és debuggolás,
- frontend megoldások és reszponzív elrendezések kialakítása,
- TypeScript típushibák javítása,
- route guard-ok és jogosultságkezelési logikák megtervezése,
- dokumentációs és szakdolgozati szövegek nyelvi javítása,
- fejezetstruktúrák és szakmai megfogalmazások kialakítása.

Az AI által generált megoldásokat minden esetben manuálisan ellenőriztem és validáltam. Az AI kimenetét nem kész megoldásként, hanem javaslatként kezeltem. A generált kódokat saját fejlesztői tudásom alapján módosítottam, futtatással és manuális teszteléssel ellenőriztem, valamint összevettem az Angular^[1] és Firebase^[2] hivatalos dokumentációival. Különösen fontos volt a biztonsági és jogosultságkezelési megoldások

ellenőrzése, mivel ezek hibás működése komoly problémákat okozhatott volna az alkalmazásban.

A fejlesztés során több olyan eset is előfordult, amikor az AI által javasolt megoldás hibásnak vagy nem megfelelőnek bizonyult. Például egyes Firestore Security Rules javaslatok túl széles jogosultságokat biztosítottak bizonyos műveletekhez, amelyeket manuálisan kellett pontosítani annak érdekében, hogy a felhasználók kizárólag saját adataikhoz férhessenek hozzá. Szintén előfordult, hogy Angular^[1] Observable alapú megoldások esetében az AI nem megfelelő adatfolyam-kezelést javasolt, amely memóriaszivárgáshoz vagy hibás állapotfrissítéshez vezethetett volna. Ezeket a problémákat saját teszteléssel és hibakereséssel azonosítottam és javítottam.

A validálás több módszerrel történt. A kódok helyességét futtatással, manuális teszteléssel és különböző felhasználói szerepkörökkel végzett ellenőrzésekkel vizsgáltam. Emellett rendszeresen használtam az Angular^[1] és Firebase^[2] hivatalos dokumentációit referenciaként. Bizonyos esetekben ugyanarra a problémára több AI eszközzel is kértem javaslatot, majd összehasonlítottam az eredményeket.

A fejlesztés során voltak olyan részek is, ahol szándékosan nem használtam mesterséges intelligencia alapú segítséget. A rendszer architektúrájának végleges kialakítása, az adatmodell megtervezése, a jogosultsági rendszer logikai felépítése, valamint a funkcionális döntések saját fejlesztői munkaként készültek el. A szakdolgozat gondolati felépítését, a projekt értékelését és a továbbfejlesztési lehetőségek meghatározását szintén saját önálló munkával készítettem el, mivel ezek személyes tapasztalatokon és fejlesztői döntéseken alapultak.

A mesterséges intelligencia használata jelentősen felgyorsította a fejlesztési folyamatot, különösen hibakeresés, dokumentáció értelmezése és bizonyos technológiák megismerése során. Ugyanakkor több esetben félrevezető vagy pontatlan megoldásokat is adott, ezért szükségessé vált a folyamatos ellenőrzés és kritikus szemlélet alkalmazása. A projekt során megtapasztaltam, hogy az AI eszközök hatékony támogatást nyújthatnak a fejlesztésben, azonban kizárólag megfelelő szakmai kontroll mellett használhatók biztonságosan.

Összességében a mesterséges intelligencia alapú eszközök jelentős segítséget nyújtottak a projekt elkészítésében, azonban a végső döntések, a rendszer kialakítása, a hibajavítások és az elkészült alkalmazás működéséért teljes mértékben saját fejlesztői felelősséget vállaltok.

12. Tesztelés és validáció

A PecaPont webalkalmazás fejlesztése során kiemelt figyelmet fordítottam a rendszer működésének ellenőrzésére és validálására. A tesztelés célja az volt, hogy az alkalmazás funkciói stabilan, hibamentesen és a meghatározott jogosultsági szabályoknak megfelelően működjenek különböző felhasználói szerepkörök esetén is.

A rendszer tesztelése elsősorban nagy mennyiségű manuális funkcionális teszteléssel történt. A fejlesztés során folyamatosan végigmentem az alkalmazás főbb folyamataiban szereplő műveleteken, és különböző felhasználói szerepkörökkel ellenőriztem a rendszer működését. A manuális tesztelés lehetővé tette a felhasználói élmény, a navigáció, valamint az egyes funkciók gyakorlati működésének részletes ellenőrzését.

A felhasználói hitelesítés tesztelése során ellenőriztem a regisztrációs és bejelentkezési folyamat működését. Vizsgálatra került az e-mail és jelszó alapú regisztráció, a Google alapú hitelesítés, valamint a hibás adatok kezelésének működése. Teszteltem továbbá a kijelentkezési folyamatot és a felhasználói állapot megfelelő frissülését is.

A tavak kezeléséhez kapcsolódó funkciók esetében ellenőriztem a tavak listázását, a keresési és szűrési funkciók működését, valamint az egyes tavak részletes adatainak megjelenítését. Adminisztrátori és tókezelői jogosultsággal teszteltem a tavak adatainak módosítását és kezelését is.

A foglalási rendszer validálása során többször végigteszteltem a teljes foglalási folyamatot. Ellenőriztem új foglalások létrehozását, a foglalási adatok mentését, valamint a foglalások megjelenítését a felhasználói profil oldalon. A tókezelői felületen teszteltem a foglalások jóváhagyását és elutasítását, valamint a státuszok megfelelő frissítését a Firestore adatbázisban.

A hírek és versenyek kezelésénél tesztelésre került az új tartalmak létrehozása, szerkesztése és megjelenítése. Az újságíró szerepkörrel rendelkező felhasználók esetében ellenőriztem a hírszerkesztő funkciókat, míg a szervező szerepkörrel a versenyek létrehozását és kezelését vizsgáltam.

A galéria funkciók tesztelése során ellenőriztem a képfeltöltés működését, a Firebase^[2] Storage-ba történő mentést, valamint a feltöltött fogások adatainak megfelelő megjelenítését. Vizsgáltam a hibás vagy hiányos adatok kezelését, valamint a feltöltött tartalmak megjelenését különböző eszközökön is.

A jogosultságkezelés ellenőrzése a validáció egyik legfontosabb része volt. Több alkalommal teszteltem, hogy jogosulatlan felhasználók ne férhessenek hozzá adminisztrátori, hírszerkesztői vagy tókezelői oldalakhoz. Ellenőriztem az Angular^[1] route guard-ok működését, valamint validáltam a Firestore Security Rules szabályait annak érdekében, hogy adatbázis szinten is tiltva legyenek az illetéktelen műveletek és módosítások.

A manuális tesztelés mellett az alkalmazás automatizált validációs mechanizmusokat is használ. Az Angular^[1] űrlapvalidációi ellenőrzik a felhasználók által megadott adatok helyességét és teljességét, míg a Firebase^[2] biztonsági szabályai backend oldalon biztosítják a jogosultság alapú hozzáférés-kezelést.

A rendszer tesztelése különböző képernyőméreteken és eszközökön is megtörtént. Az alkalmazást asztali számítógépen, tableten és mobil eszközökön is ellenőriztem annak érdekében, hogy a reszponzív megjelenítés megfelelően működjön.

A tesztelések eredményei alapján a rendszer stabilan működik, a jogosultságkezelés megfelelően elkülöníti az egyes szerepkörök funkcióit, valamint a főbb felhasználói folyamatok hibamentesen végrehajthatók.

13. Tapasztalatok, továbbfejlesztési lehetőségek

A szakdolgozat készítésének megkezdése előtt már eldöntöttem, hogy a webalkalmazás fejlesztése során az Angular^[1] keretrendszert és a Firebase^[2] felhőalapú szolgáltatásait fogom használni. Ennek egyik oka az volt, hogy korábban már szereztem tapasztalatot Angular^[1] alapú fejlesztésben, ezért szerettem volna tovább mélyíteni tudásomat a keretrendszer újabb verzióiban megjelent megoldásokkal kapcsolatban. Különösen érdekesnek találtam az Angular^[1] standalone komponens alapú felépítését, valamint a signal alapú állapotkezelési lehetőségeket, amelyek jelentősen egyszerűsítették az alkalmazás felépítését és a komponensek közötti adatkezelést.

A backend technológia kiválasztásakor fontos szempont volt, hogy a rendszer könnyen integrálható legyen Angular^[1] környezetben, valamint gyors fejlesztést és egyszerű üzemeltetést biztosítson. A Firebase^[2] szolgáltatásai – különösen a Firestore, Authentication, Cloud Storage és Hosting – lehetővé tették, hogy hagyományos backend szerver fejlesztése nélkül is teljes értékű webalkalmazást hozzak létre. A Firebase^[2] használata jelentősen leegyszerűsítette az autentikációt, az adatkezelést és a hosztolást megvalósítását.

A fejlesztési folyamat kezdetén elsőként a projekt struktúráját és az alapvető funkcionalitásokat terveztem meg. Ezt követően elkészítettem az első képernyőterveket, amelyek során kiemelt figyelmet fordítottam a felhasználói élményre és az átlátható kezelőfelület kialakítására. A tervezés során fokozatosan bővült a funkciók köre is, mivel a fejlesztés közben újabb ötletek és igények merültek fel.

A fejlesztés során különösen érdekes feladatot jelentett a jogosultságkezelési rendszer kialakítása. A kezdeti felhasználói és adminisztrátori szerepkörök mellett később további szerepkörök kerültek bevezetésre, például az újságíró, a szervező és a tókezelő szerepkör. Ezek megvalósítása során lehetőség nyílt komplexebb szerepkör alapú hozzáférés-kezelési mechanizmusok kialakítására mind frontend, mind backend oldalon. A Firebase^[2] Firestore biztonsági szabályainak konfigurálása új tapasztalatot jelentett számomra, azonban a rendszer rugalmassága lehetővé tette a részletes jogosultságkezelés kialakítását.

A projekt fejlesztése során a webalkalmazás funkcionalitása jelentősen kibővült az eredeti elképzelésekhez képest. A kezdeti információs és közösségi funkciók mellett megvalósításra került a foglalási rendszer, az értékelési rendszer, valamint a tókezelői felület is. A foglalási rendszer különösen összetett funkciónak bizonyult, mivel több szereplő – felhasználók és tókezelők – közötti interakciót és állapotkezelést kellett megvalósítani.

A fejlesztési folyamat során a GitHub^[4] verziókezelő rendszer folyamatos használata nagy segítséget nyújtott a projekt strukturált kezelésében. A verziókövetés lehetővé tette a módosítások nyomon követését és a fejlesztés biztonságosabb végzését. Emellett a ChatGPT^[5] és más mesterséges intelligencia alapú eszközök használata is hozzájárult a fejlesztés gyorsításához, különösen hibakeresés, dokumentációk értelmezése és egyes technikai problémák megoldása során.

A fejlesztés során számos új technológiával és megoldással sikerült megismerkednem. Különösen hasznos tapasztalatot jelentett a Firebase^[2] alapú valós idejű adatkezelés, a role based access control megvalósítása, valamint a reszponzív felhasználói felületek kialakítása.

A projekt ugyanakkor több továbbfejlesztési lehetőséget is magában hordoz. A jövőben megvalósítható lenne például a többnyelvű támogatás, amely lehetővé tenné az alkalmazás nemzetközi használatát is. További fejlesztési lehetőséget jelenthet push értesítések bevezetése a foglalások és események kezeléséhez, valamint valós idejű üzenetküldési funkciók kialakítása a felhasználók között.

A rendszer később mobilalkalmazás formájában is továbbfejleszthető lenne, például Ionic vagy Angular^[1] alapú hibrid alkalmazás segítségével. Bár a jelenlegi webalkalmazás reszponzív kialakításának köszönhetően mobil eszközökön is jól használható, egy natívabb mobilalkalmazás gyorsabb és kényelmesebb felhasználói élményt biztosíthatna.

Összességében a szakdolgozat elkészítése során sikerült egy olyan modern webalkalmazást létrehoznom, amely ötvözi a közösségi funkciókat, az információkezelést és a foglalási rendszert. A fejlesztés során szerzett tapasztalatok jelentősen hozzájárultak szakmai tudásom fejlődéséhez, különösen modern frontend technológiák, felhőalapú backend szolgáltatások és webalkalmazás-architektúrák területén.

Irodalomjegyzék

1. Angular: <https://angular.dev/> (Utolsó megtekintés: 2026.05.18.)
2. Google Firebase: <https://firebase.google.com/> (Utolsó megtekintés: 2026.05.18.)
3. Visual Studio Code: <https://code.visualstudio.com/> (Utolsó megtekintés: 2026.05.18.)
4. GitHub: <https://github.com/> (Utolsó megtekintés: 2026.05.18.)
5. ChatGPT - OpenAI: <https://chatgpt.com/> (Utolsó megtekintés: 2026.05.18.)
6. Gemini – Google AI: <https://gemini.google.com> (Utolsó megtekintés: 2026.05.18.)
7. AngularFire: <https://github.com/angular/angularfire> (Utolsó megtekintés: 2026.05.18.)
8. Draw.io: <https://www.diagrams.net> (Utolsó megtekintés: 2026.05.18.)
9. Horgasz helyek.hu: <https://horgasz helyek.hu> (Utolsó megtekintés: 2026.05.18.)
10. Vampire Fishing Trips:
<https://www.vampirebitealarm.hu/pages/applikacio?shpxid=74e8e4ef-4978-4735-8b89-96f96e578255> (Utolsó megtekintés: 2026.05.18.)
11. FishMap: <https://fishmap.eu/> (Utolsó megtekintés: 2026.05.18.)

Nyilatkozat

Alulírott Szabó Patrik Péter programtervező informatikus szakos hallgató, kijelentem, hogy a dolgozatomat a Szegedi Tudományegyetem, Informatikai Intézet Szoftverfejlesztés Tanszékén készítettem, BSc diploma megszerzése érdekében.

Kijelentem, hogy a dolgozatot más szakon korábban nem védtem meg, saját munkám eredménye, és csak a hivatkozott forrásokat (szakirodalom, eszközök, stb.) használtam fel. Tudomásul veszem, hogy szakdolgozatomat / diplomamunkámat a Szegedi Tudományegyetem Diplomamunka Repoitóriumban tárolja.

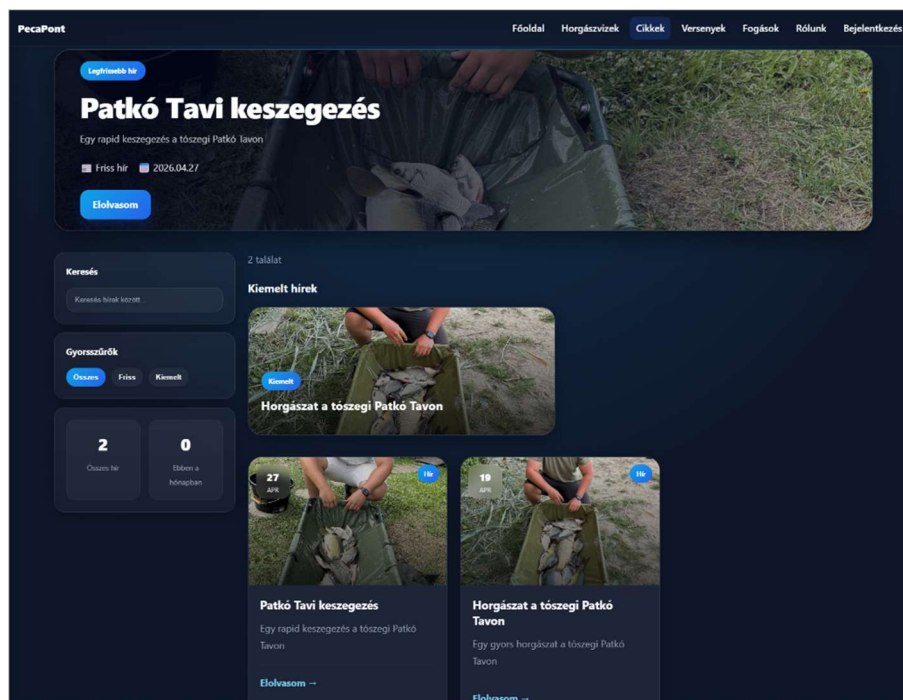
2026.05.18.

Elektronikus mellékletek

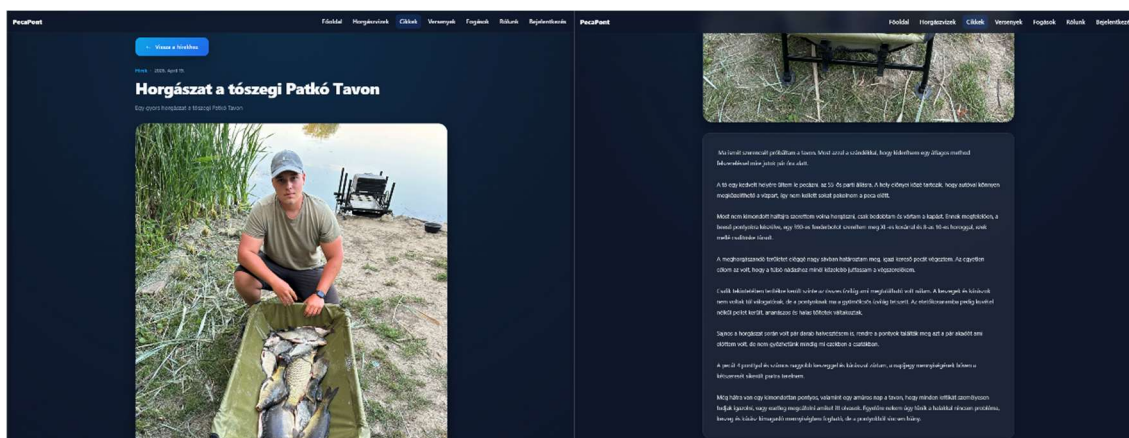
A szakdolgozathoz kapcsolódó forráskód és a futtatható webalkalmazás az alábbi elérhetőségeken található.

1. A webalkalmazás GitHub elérhetősége: <https://github.com/szpatrik55/PecaPont>
2. A webalkalmazás hosztolt linkje: <https://pecapont--pecapont-50489.europe-west4.hosted.app>
3. Végleges branch: main
4. Végleges commit azonosító: 897f2ea24f5ebd31b24a8dd566105ecdab610488

Mellékletek

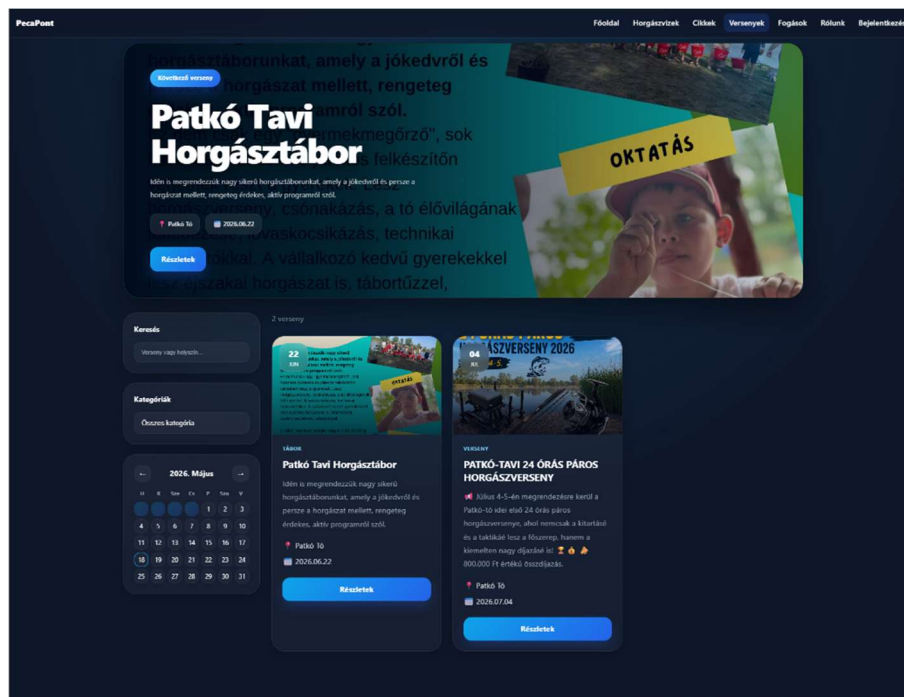


1. melléklet A webalkalmazás Hírek oldalának megjelenése

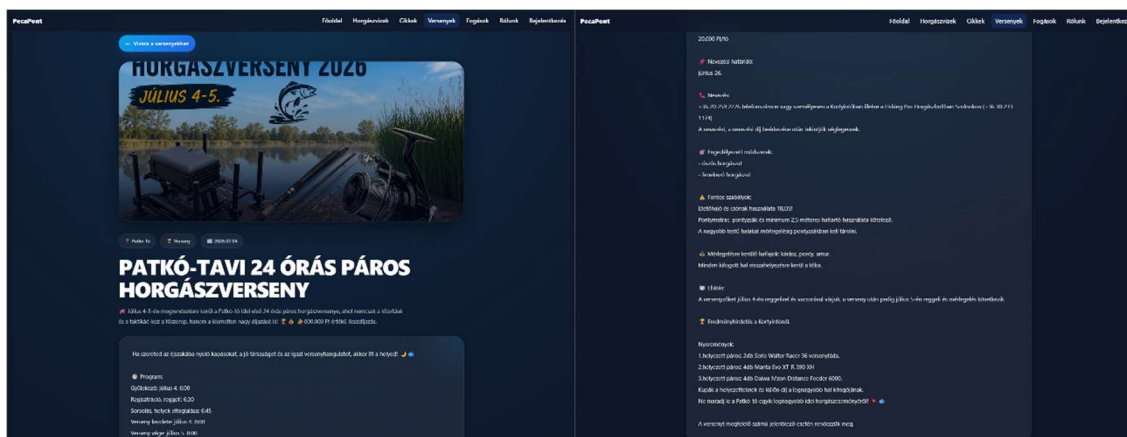


2. melléklet A webalkalmazás Hír részletes oldalának megjelenése

PecaPont: Horgászhely foglaló és információ szerző webalkalmazás

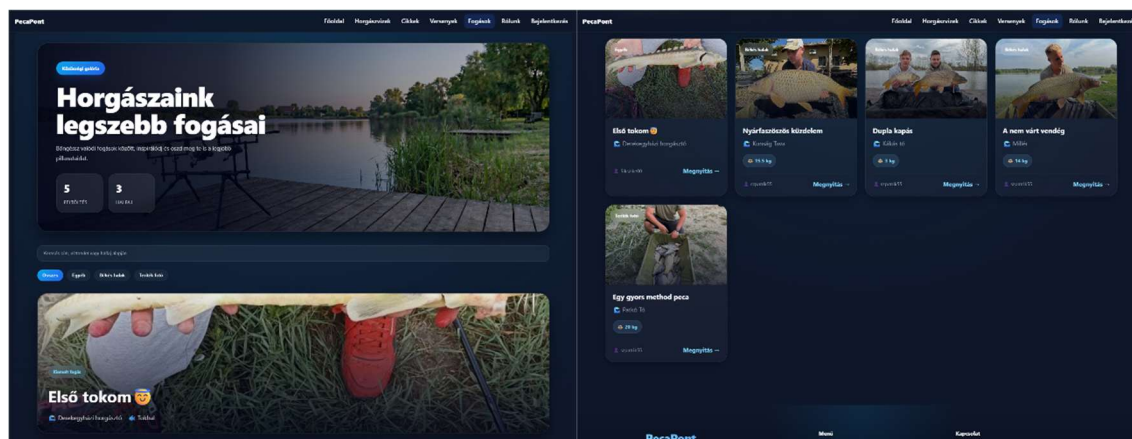


3. melléklet A webalkalmazás Versenyek oldalának megjelenése

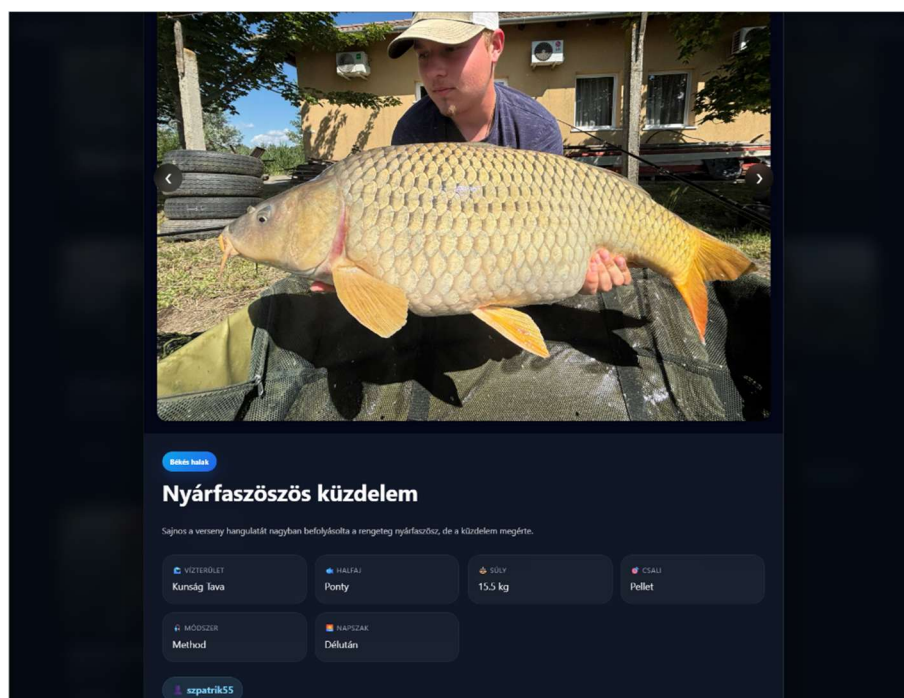


4. melléklet A webalkalmazás Verseny részletes oldalának megjelenése

PecaPont: Horgászhely foglaló és információ szerző webalkalmazás

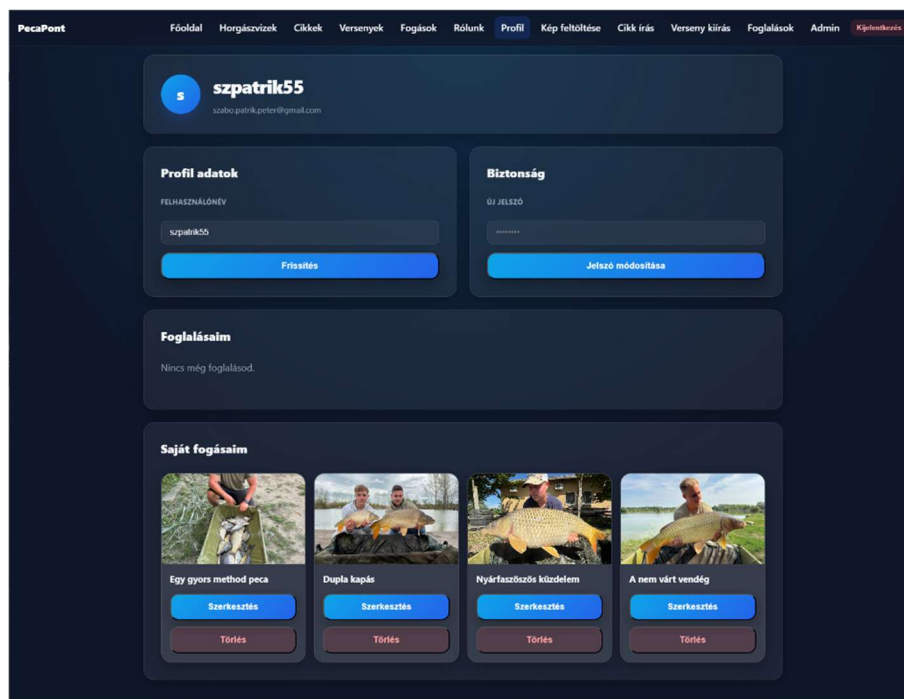


5. melléklet A webalkalmazás Galéria oldalának megjelenése

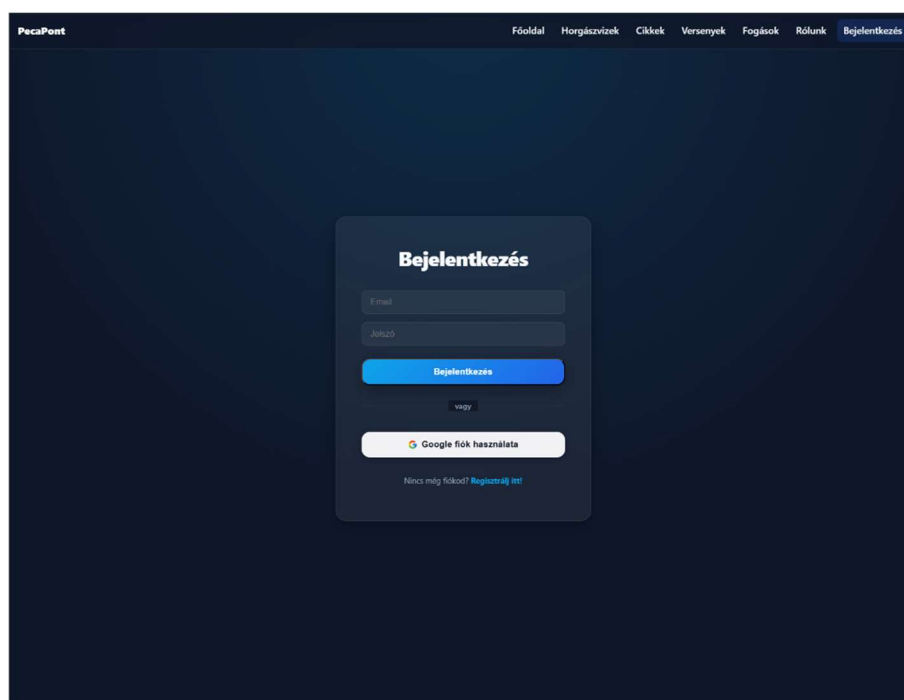


6. melléklet A webalkalmazás Fogás részlet oldalának megjelenése

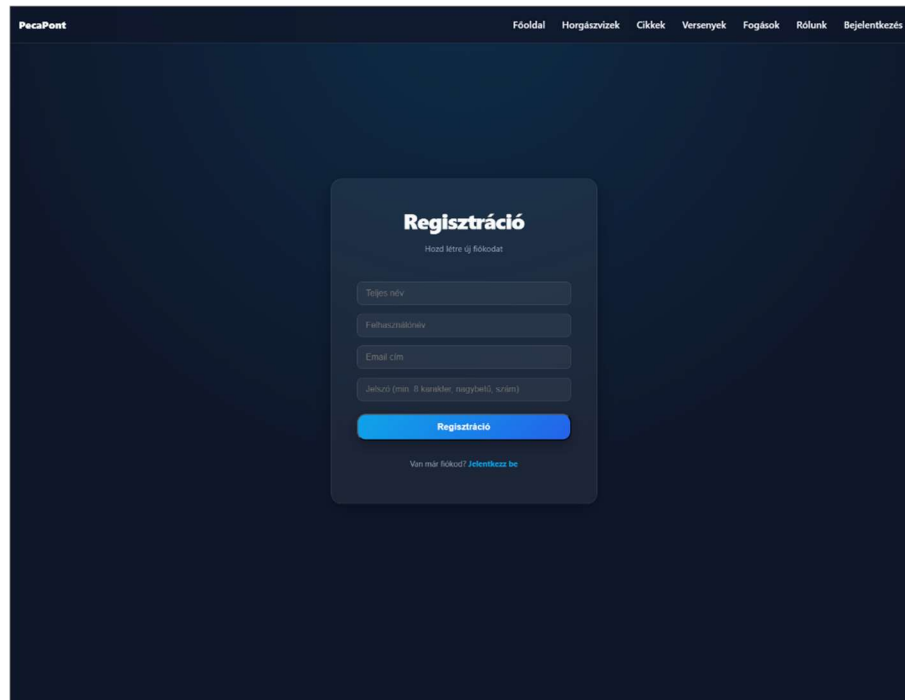
PecaPont: Horgászhely foglaló és információ szerző webalkalmazás



7. melléklet A webalkalmazás Profil oldalának megjelenése



8. melléklet A webalkalmazás Bejelentkezés oldalának megjelenése



9. melléklet A webalkalmazás Regisztráció oldalának megjelenése

- *app:*
A webalkalmazás gyökér komponense, a RouterOutlet segítségével dinamikusan tölti be az aktuálisan megjelenítendő oldalt.
- *home:*
A webalkalmazás főoldala, amely áttekintést nyújt a rendszer főbb funkcióiról és tartalmairól. Ez tölt be bejelentkezés és regisztráció, valamint kijelentkezés után is.
- *regisztracio:*
A webalkalmazás regisztrációs oldala. A felhasználók számára biztosít lehetőséget új fiók létrehozására, email és jelszó páros megadásával, valamint Google fiók használatával.
- *bejelentkezés:*
A webalkalmazás bejelentkezés oldala. Itt tudnak a felhasználók a felületre bejelentkezni.
- *tavak:*
A webalkalmazás Tavak oldala. A horgászhelyek listáját jeleníti meg, ahol a felhasználók böngészhetnek és szűrhetnek a különböző tavak között.
- *to-reszletek:*
A webalkalmazás egy adott tavát részletesen megjelenítő oldal. Egy kiválasztott

horgászhely részletes adatait jeleníti meg, vízterület méretét, szabályokat, ajánlott módszereket, felhasználói értékeléseket, valamint a foglalási lehetőségeket.

- *to-hozzaad:*
A webalkalmazás tó hozzáadása oldala. Az admin szerepkörrel rendelkező felhasználók számára nyújt lehetőséget új vízterületek felvételéhez.
- *hirek:*
A webalkalmazás Cikk oldala. A felhasználók itt tudnak böngészni és keresni a cikkek között.
- *hir-reszletek:*
A webalkalmazás egy adott cikkjét részletező oldal. Egy adott cikk részletes megjelenítéséért felel, megjelennek a részletek és a teljes tartalom.
- *hir-szerkeszto:*
A webalkalmazás cikkek írásáért felelős oldala. Az admin, valamint újságíró szerepkörrel rendelkező felhasználók érik el és itt tudnak cikkeket közzétenni a felhasználók számára.
- *versenyek:*
A webalkalmazás Versenyek oldala. Az aktuális horgászversenyek listáját jeleníti meg. A felhasználók kereshetnek és rendezhetik a tartalmat.
- *verseny-reszletek:*
A webalkalmazás egy adott versenyét megjelenítő oldal. A verseny részletes leírását, időpontját és fontos információt jeleníti meg.
- *verseny-szerkeszto:*
A webalkalmazás versenyek publikálására szolgáló oldala. Az admin, valamint szervező szerepkörrel rendelkező felhasználók érhetik el és itt tudnak új versenyeményeket közzétenni a felhasználók számára.
- *manager-dashboard*
A webalkalmazás tőkezelői felülete. A tőkezelő szerepkörrel rendelkező felhasználók számára biztosít áttekintő oldalt, ahol kártyás nézetben jelennek meg az általuk kezelt tavak. A komponens lehetőséget biztosít az egyes tavakhoz tartozó foglalások megtekintésére és kezelésére.
- *manager-bookings:*
A webalkalmazás tőkezelői foglaláskezelő oldala. Egy adott tóhoz tartozó foglalások listáját jeleníti meg, ahol a tőkezelő megtekintheti a foglalások részleteit,

valamint kezelheti azok állapotát, például jóváhagyhatja vagy elutasíthatja a foglalásokat.

- *fogasok:*

A webalkalmazás Fogások oldala. A felhasználók itt láthatják egymás fogásait, képeit, amelyekhez csatolhatnak rövid- és hosszú leírást, a fogás típusát és súlyát vagy hosszát.

- *rolunk:*

A webalkalmazás Rólunk oldala. Egy bemutatkozó oldal, ahol a felhasználók jobban megismerhetik az oldal koncepcióját.

- *profil:*

A webalkalmazás Profil oldala. Itt tudják a felhasználók módosítani az adataikat, kezelni a feltöltött fogásaikat, megtekinteni és lemondani foglalásaikat, valamint törölni a fiókjukat.

- *admin-dashboard:*

A webalkalmazás admin panele. Az admin szerepkörrel rendelkező felhasználók részére készült áttekintő felület, amely statisztikai adatokat jelenít meg a rendszer működéséről, például a felhasználók számát, a horgászhelyek számát, valamint az aktív és új felhasználók számát. A dashboard segíti az adminisztrátort a rendszer állapotának nyomon követésében.

- *admin-layout:*

A webalkalmazás admin panelének váza. Az admin felület alapstruktúráját biztosító komponens, amely az adminisztrációs oldalak közös elrendezéséért felel.

- *admin-users:*

A webalkalmazás admin panelének felhasználók kezelésére szolgáló része. Az admin szerepkörrel rendelkező felhasználók számára lehetőséget biztosít a felhasználók kezelésére. A komponens megjeleníti a felhasználók listáját, és lehetőséget ad a szerepkörök módosítására, ezáltal támogatva a jogosultságkezelést a rendszerben.

A PecaPont webalkalmazásban az alábbi újrafelhasználható komponenseket hoztam létre:

- *navbar:*

A webalkalmazás fő navigációs eleme. Lehetővé teszi az oldalak közötti navigációt. A komponens minden oldalon megjelenik, biztosítva az egységes felhasználói élményt.

- *footer:*

A webalkalmazás alsó részén megjelenő komponens. Egységes információkat tartalmaz, a komponens minden oldalon megjelenik, biztosítva a konzisztens felhasználói felületet.

10. melléklet A webalkalmazás komponenslistája

- *admin:*

Az admin service az adminisztrációs funkciók megvalósításáért felelős szolgáltatás. A szolgáltatás a Firebase^[2] Firestore adatbázissal kommunikál, és lehetővé teszi a felhasználók és horgászhelyek adatainak lekérését és kezelését. A szolgáltatás segítségével lekérhetők a felhasználók adatai, módosíthatók a felhasználói szerepkörök, valamint törölhetők a felhasználók. Emellett különböző statisztikai adatok lekérésére is lehetőséget biztosít, például a felhasználók és horgászhelyek számának meghatározására. További funkcióként képes meghatározni az új felhasználók számát az elmúlt időszakban, az aktív felhasználók számát, valamint napi bontásban megjeleníteni a regisztrációk alakulását. Ezek az adatok az adminisztrációs felületen jelennek meg, segítve a rendszer állapotának áttekintését.

- *auth:*

Az auth service a felhasználók hitelesítéséért és jogosultságkezeléséért felelős szolgáltatás. A szolgáltatás a Firebase^[2] Authentication és a Firestore adatbázis együttes használatával valósítja meg a felhasználók kezelését. A szolgáltatás figyeli az aktuális autentikációs állapotot, és ennek megfelelően lekéri a felhasználóhoz tartozó adatokat a Firestore adatbázisból. Ez lehetővé teszi, hogy az alkalmazás a bejelentkezett felhasználóhoz tartozó kiegészítő információkat, például szerepkört és profiladatokat is kezelje. A szolgáltatás támogatja az e-mail és jelszó alapú regisztrációt és bejelentkezést, valamint a Google fiókkal történő hitelesítést is. A regisztráció során a felhasználó adatai bekerülnek az adatbázisba, beleértve a létrehozás időpontját és a felhasználó szerepkörét. A bejelentkezés során a szolgáltatás frissíti a felhasználó utolsó belépésének időpontját, amely később statisztikai célokra is felhasználható. A szolgáltatás továbbá kezeli a felhasználói jogosultságokat is. Külön Observable értékeken keresztül biztosítja annak ellenőrzését, hogy az adott felhasználó rendelkezik-e adminisztrátori vagy szerkesztői, szervezői jogosultságokkal. Ez lehetővé teszi a különböző funkciókhoz való hozzáférés szabályozását az alkalmazásban.

- *gallery:*

A gallery service a felhasználók által feltöltött képek és galéria tartalmak kezeléséért felelős szolgáltatás. A szolgáltatás a Firebase^[2] Cloud Storage és a Firestore adatbázis együttes használatával valósítja meg a képfeltöltést és a kapcsolódó adatok tárolását. A szolgáltatás lehetővé teszi képfájlok feltöltését a felhőbe, majd a feltöltött fájlok elérési útjának (URL) lekérését. Ez az URL később felhasználható a képek megjelenítésére a webalkalmazásban. Emellett a szolgáltatás biztosítja a galéria bejegyzések létrehozását is, amelyek tartalmazhatják a feltöltött képekhez kapcsolódó adatokat. Ezek az adatok a Firestore adatbázisban kerülnek tárolásra. A szolgáltatás aszinkron műveleteket használ, és az Angular^[1] zónakezelésének segítségével biztosítja, hogy a felhasználói felület megfelelően frissüljön a feltöltési folyamat során.

- *news:*

A news service a hírek és cikkek kezeléséért felelős szolgáltatás. A szolgáltatás a Firebase^[2] Firestore adatbázist és a Cloud Storage szolgáltatást használja a tartalmak tárolására és kezelésére. A szolgáltatás lehetővé teszi képek feltöltését a hírekhez, amelyeket a rendszer a felhőben tárol, majd a hozzájuk tartozó elérési út (URL) visszaadásra kerül. Ez az URL később felhasználható a hírek megjelenítése során. A szolgáltatás támogatja új hírek létrehozását, amelyek a Firestore adatbázisban kerülnek tárolásra. A hírek létrehozásakor a rendszer automatikusan rögzíti a létrehozás időpontját, amely lehetővé teszi az adatok időrendi rendezését. A szolgáltatás továbbá biztosítja az összes hír lekérdezését, valamint az egyedi hírek elérését azonosító alapján. A hírek lekérdezése időrendi sorrendben történik, így a legfrissebb tartalmak jelennek meg elsőként a felhasználói felületen.

- *booking:*

A booking service a horgász hely foglalások kezeléséért felelős szolgáltatás. A szolgáltatás a Firebase^[2] Firestore adatbázissal kommunikál, és lehetővé teszi a foglalások létrehozását, lekérdezését és módosítását. A felhasználók új foglalásokat hozhatnak létre, valamint megtekinthetik saját foglalásaikat és azok állapotát. A szolgáltatás támogatja a tökezelői funkciókat is, amelyek segítségével a tökezelők lekérhetik az általuk kezelt tavakhoz tartozó foglalásokat, valamint módosíthatják azok státuszát, például jóváhagyhatják vagy elutasíthatják a

foglalási kérelmeket. A foglalások időrendi sorrendben kerülnek lekérdezésre, ezzel támogatva az átlátható kezelésüket.

- events:

Az events service a horgászversenyek és események kezeléséért felelős szolgáltatás. A szolgáltatás a Firebase^[2] Firestore adatbázist használja az események adatainak tárolására és kezelésére. A szolgáltatás lehetővé teszi új események létrehozását, meglévő események módosítását, valamint az események lekérdezését és megjelenítését. Az események időrendi sorrendben kerülnek lekérdezésre, így a felhasználók a legaktuálisabb versenyeket láthatják elsőként.

- lake:

A lake service a horgász helyek adatainak kezeléséért felelős szolgáltatás. A szolgáltatás biztosítja a tavak adatainak lekérdezését, létrehozását és módosítását a Firebase^[2] Firestore adatbázis segítségével. A szolgáltatás támogatja a tavak közötti keresést és szűrést, valamint lehetőséget biztosít az egyes tavak részletes adatainak megjelenítésére. Emellett a tókezelő szerepkörrel rendelkező felhasználók számára biztosítja a saját kezelésük alatt álló tavak adatainak kezelését is.

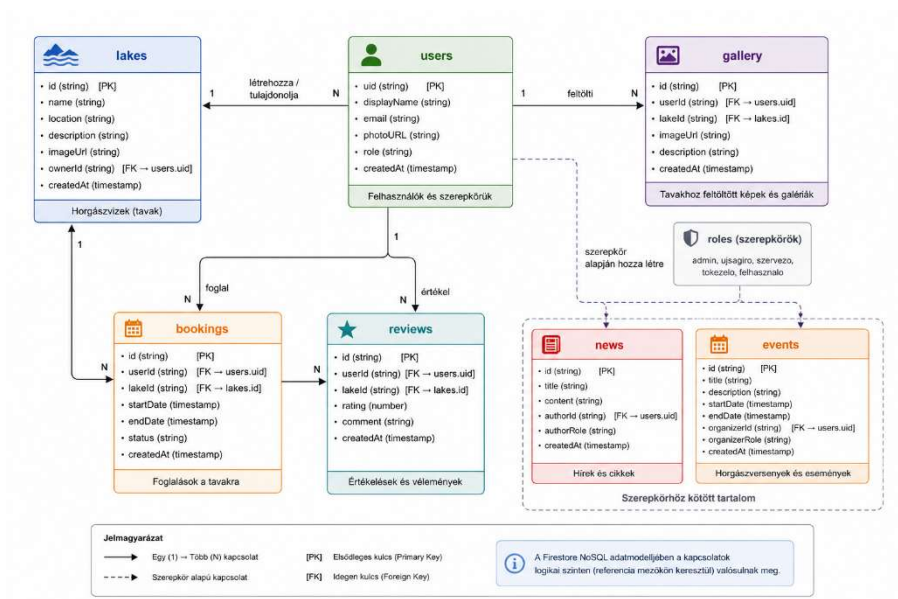
- review:

A review service a felhasználói értékelések és vélemények kezeléséért felelős szolgáltatás. A szolgáltatás a Firebase^[2] Firestore adatbázist használja az értékelések tárolására és lekérdezésére. A felhasználók lehetőséget kapnak arra, hogy értékeléseket és szöveges véleményeket írjanak az egyes horgász helyekhez. A szolgáltatás biztosítja az értékelések létrehozását, lekérdezését és megjelenítését, ezáltal támogatva a közösségi alapú információmegosztást és a horgász helyek összehasonlíthatóságát.

11. melléklet A webalkalmazás service listája

- *Booking:*
A foglalásokat reprezentáló interfész. Tartalmazza a foglaláshoz kapcsolódó adatokat, például a foglaló felhasználó azonosítóját és adatait, a kiválasztott tó adatait, a foglalás időintervallumát, az árakat, valamint a foglalás aktuális állapotát. Az interfész kezeli a foglalások státuszait is, például a jóváhagyás alatt, jóváhagyva, elutasítva és törölve állapotokat.
- *EventItem:*
A horgászversenyeket és eseményeket reprezentáló interfész. Tartalmazza az esemény nevét, rövid és részletes leírását, helyszínét, időpontját, kategóriáját, valamint az eseményhez kapcsolódó kép elérési útját.
- *GalleryPost:*
A felhasználók által feltöltött fogásokat és galéria bejegyzéseket reprezentáló interfész. Tartalmazza a fogás címét, leírását, a horgászhely nevét, a halfaj adatait, valamint a fogáshoz kapcsolódó további információkat, például a súlyt, hosszúságot, alkalmazott módszert és csalit. Az interfész tartalmazza továbbá a feltöltött kép elérési útját, valamint a feltöltő felhasználó adatait is.
- *Lake:*
A horgászhelyeket reprezentáló interfész. Tartalmazza a tavak alapadatait, például a nevét, települését, címét, leírását és típusát. Emellett tárolja a horgászhelyhez kapcsolódó adatokat, például a terület méretét, vízmélységet, férőhelyek számát, jegyárakat, halfajokat, szabályokat és ajánlott módszereket. Az interfész kezeli a tókezelő rendszerhez kapcsolódó adatokat is, például a tókezelő azonosítóját és nevét.
- *Review:*
A felhasználói értékeléseket reprezentáló interfész. Tartalmazza az értékelést létrehozó felhasználó adatait, az adott pontszámot, a szöveges véleményt, valamint az értékelés létrehozásának időpontját. Az interfész lehetővé teszi a közösségi alapú visszajelzések kezelését és megjelenítését a rendszerben.

PecaPont: Horgászhely foglaló és információ szerző webalkalmazás



13. melléklet a webalkalmazás adatmodell diagramja